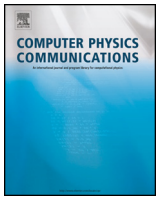
Computer Physics Communications 287 (2023) 108700

Contents lists available at ScienceDirect

Computer Physics Communications

journal homepage: www.elsevier.com/locate/cpc



HORSES3D: A high-order discontinuous Galerkin solver for flow simulations and multi-physics applications ☆,☆☆

E. Ferrer [a,b], G. Rubio [a,b,*], G. Ntoukas [a], W. Laskowski [a], O.A. Mariño [a], S. Colombo [a], A. Mateo-Gabín [a], H. Marbona [a], F. Manrique de Lara [a], D. Huergo [a], J. Manzanero [e], A.M. Rueda-Ramírez [c], D.A. Kopriva [d], E. Valero [a,b]

[a] ETSIAE-UPM-School of Aeronautics, Universidad Politécnica de Madrid, Madrid, Spain
[b] Center for Computational Simulation, Universidad Politécnica de Madrid, Madrid, Spain
[c] University of Cologne, Division of Mathematics, Köln, Germany
[d] Department of Mathematics, Florida State University and Computational Science Research Center, San Diego State University, USA
[e] Airbus Defence and Space, Madrid, Spain

ARTICLE INFO

Article history:
Received 24 June 2022
Received in revised form 3 February 2023
Accepted 11 February 2023
Available online 24 February 2023

Dataset link: https://github.com/loganoz/horses3d

Keywords:
Navier-Stokes
Discontinuous Galerkin
High-order method
Multiphase
Aeroacoustics
Immersed boundary method

ABSTRACT

We present the latest developments of our **H**igh-**O**rder **S**pectral **E**lement **S**olver (HORSES3D), an open source high-order discontinuous Galerkin framework, capable of solving a variety of flow applications, including compressible flows (with or without shocks), incompressible flows, various RANS and LES turbulence models, particle dynamics, multiphase flows, and aeroacoustics. We provide an overview of the high-order spatial discretisation (including energy/entropy stable schemes) and anisotropic p-adaptation capabilities. The solver is parallelised using MPI and OpenMP showing good scalability for up to 1000 processors. Temporal discretisations include explicit, implicit, multigrid, and dual time-stepping schemes with efficient preconditioners. Additionally, we facilitate meshing and simulating complex geometries through a mesh-free immersed boundary technique. We detail the available documentation and the test cases included in the GitHub repository.

Program summary

Program Title: HORSES3D
CPC Library link to program files: https://doi.org/10.17632/738py2shk4.1
Developer's repository link: https://github.com/loganoz/horses3d.
Licensing provisions: MIT
Programming language: Fortran 2008
External routines/libraries: METIS, MPI, HDF5, MKL, PETSc (all are optional).
Nature of problem: HORSES3D is a high-order discontinuous Galerkin framework, capable of solving a variety of flow applications, including compressible flows (with or without shocks), incompressible flows, various RANS and LES turbulence models, particle dynamics, multiphase flows, and aeroacoustics.
Solution method: high-order discontinuous Galerkin Spectral Element (DGSEM) and explicit/implicit time-steppers.
Additional comments including restrictions and unusual features: HORSES3D is a multiphysics environment where the compressible Navier-Stokes equations, the incompressible Navier–Stokes equations, the Cahn–Hilliard equation and entropy–stable variants are solved. Arbitrary high–order, p–anisotropic discretisations are used, including static and dynamic p–adaptation methods (feature-based and truncation error-based). Explicit and implicit time-steppers for steady and time-marching solutions are available, including efficient multigrid and preconditioners. Numerical and analytical Jacobian computations with a colouring algorithm have been implemented. Multiphase flows are solved using a diffuse interface model: Navier–Stokes/Cahn–Hilliard. Turbulent models implemented include RANS: Spalart-Allmaras and LES: Smagorinsky, Wale, Vreman; including wall models. Immersed boundary methods can be used, to avoid creating body fitted meshes. Acoustic propagation can be computed using Ffowcs-Williams and

☆ The review of this paper was arranged by Prof. N.S. Scott.

☆☆ This paper and its associated computer program are available via the Computer Physics Communications homepage on ScienceDirect (http://www.sciencedirect.com/science/journal/00104655).

* Corresponding author at: ETSIAE-UPM School of Aeronautics, Universidad Politécnica de Madrid, Pl. del Cardenal Cisneros, 3, 28040 Madrid, Spain.
E-mail address: g.rubio@upm.es (G. Rubio).

https://doi.org/10.1016/j.cpc.2023.108700

Hawkings models. HORSES3D supports curvilinear, hexahedral, conforming meshes in GMSH, HDF5 and SpecMesh/HOHQMesh format. A hybrid CPU-based parallelisation strategy (shared and distributed memory) with OpenMP and MPI is followed.

Contents

1. Introduction

To simulate and predict complex flows, the non-linear Navier-Stokes (NS) equations that govern fluid flow need to be solved numerically. Traditional simulation solvers that use low-order methods (order ≤ 2, for example, finite differences or finite volumes) have been widely adopted by industry and academia to simulate fluid flows, but have limitations when high accuracy is required [1]. Low-order numerical techniques (most commercial codes) suffer from non-negligible numerical errors (e.g., dissipative and dispersive errors) that can increase unphysical dissipation and dispersion of flow structures and provide unrealistic results.

An alternative to low order solvers is to employ high-order discretisations based on polynomial approximations of the solution. In the high-order discretisations used here, the formal order of the scheme is $p+1$, where p is the polynomial order within each mesh element. These methods are characterised by low numerical errors, and their ability to use mesh refinement through an increased number of mesh nodes (h-refinement) and/or polynomial enrichment (p-refinement) to achieve highly accurate solutions. Such high-order polynomial methods produce an exponential decay of the error for smooth enough solutions instead of the algebraic decay characteristic of low-order techniques.

Typical aircraft aerodynamic simulations require an order of 50 to 100M degrees of freedom (dof), if low-order methods are used. Using high-order methods, the overall number of dof can be reduced for a desired level of accuracy. Given a numerical scheme, the numerical error, e, is proportional to $(dof)^P$, where P is the order, see [2]. To achieve similar errors (i.e. $e_{HO} \approx e_{LO}$) between high- (HO) and low-order (LO) methods, the high-order mesh may be coarsened following: $(dof)_{HO} = \exp(P_{LO}/P_{HO} . \log[(dof)_{LO}])$. Selecting $P_{LO} = 2$ and $P_{HO} = 5$, one finds for $(dof)_{LO} = 100M$ a corresponding mesh size $(dof)_{HO} = 1.6k$; that is, a factor of 60000 decrease in dof for the same accuracy. Of course, this estimate is optimistic, since in reality small cells are required near walls to resolve boundary layers. Nevertheless, this estimate illustrates how high-order methods can enable simulations with fewer degrees of freedom (while maintaining accuracy). Additional accuracy can be achieved with a limited increase in cost, when using local anisotropic p-refinement (i.e. only particular mesh elements and directions use high-order polynomials), [3–6].

High-order numerical methods of spectral type (order $>> 2$) were first developed in the 1970s (e.g., [7]) and have typically been perceived as accurate but expensive methods, with a lack of flexibility (for complex geometries) and robustness (tendency to crash) when applied to complex turbulent flows. Although widely adopted in academia and particularly for low Reynolds numbers during the 1980s and 90s, it was not until the 2000s that these became a realistic alternative to well-established low-order tech-

niques (finite differences or finite volumes). One of the main reasons for the increasing acceptance has been the development of high-order discontinuous Galerkin (DG) variants that include interelement fluxes that enhance robustness (when compared to classic high-order continuous methods). DG has seen a remarkable recent increase in popularity in the last 10 years. DG methods were first developed by Reed & Hill in the framework of neutron transport equations [8], but remained unexploited until the late 1990s, when the method was generalised to convection-diffusion problems that are used to simulate fluid flows: see, e.g., [9,10]. Today, DG methods have been used to solve a wide variety of flows, including incompressible, compressible, or multiphase flows, but still need developments to become accepted within the industry (e.g., increased speed and robustness for turbulent flows).

DG is paving its way towards industry acceptance through research centres. For example, Airbus, Dyson, McLaren F1 or Siemens-Gamesa have collaborated with our group in various European projects to evaluate high-order methods. In these projects, aeronautical European research centres (e.g., ONERA or DLR) were also involved. DG methods are relatively mature for aeronautical applications, but require further developments to be widely adopted by industry (not necessarily aeronautical).

In this work, we summarise the main features included in our high-order DG solver HORSES3D:

- A multiphysics environment where the compressible Navier-Stokes equations, the incompressible Navier–Stokes equations, the Cahn–Hilliard equation and entropy–stable variants are solved.
- Arbitrary high–order, p–anisotropic discretisations, including static and dynamic p–adaptation methods using feature-based and truncation error estimates.
- Explicit and implicit time-steppers for steady and time-marching solutions, including efficient multigrid and preconditioners.
- Multiphase flows using a diffuse interface model: Navier–Stokes/Cahn–Hilliard.
- Turbulent models including RANS: Spalart-Allmaras and LES: Smagorinsky, Wale, Vreman; including wall models.
- Immersed boundary methods to avoid creating body fitted meshes.
- Acoustic propagation using Ffowcs-Williams and Hawkings.
- Support for curvilinear, hexahedral, conforming meshes in GMSH, HDF5 and SpecMesh/HOHQMesh format.
- Hybrid CPU-based parallelisation (shared and distributed memory) with OpenMP and MPI.

The work is structured as follows. In Sec. 2, we contextualise HORSES3D with a review of the main capabilities of available high-order academic solvers. The rest of the paper can be split in two parts. In the first part: Numerical Method (Sec. 3), we describe the original features of the solver that was first conceived to solve the compressible Navier-Stokes equations. This part includes meshing (Sec. 3.1), the compressible Navier-Stokes solver (Sec. 3.2), the discontinuous Galerkin spatial discretisation (Sec. 3.3), the possibility to include alternate physics (Sec. 3.4), the available energy/entropy stable formulations to enhance stability (Sec. 3.5), local spatial p-adaption (Sec. 3.6), the time advancement methods available (Sec. 3.7) and parallelisation benchmarks (Sec. 3.8).

In the second part: Multiphysics (Sec. 4), we detail the extended capabilities of HORSES3D, including turbulent modelling (Sec. 4.1), incompressible flows (Sec. 4.2), multiphase flows (Sec. 4.3), shock capturing (Sec. 4.4), particle dynamics (Sec. 4.5), aeroacoustics (Sec. 4.6) and immersed boundaries (Sec. 4.7).

2. Review of high-order solvers

A range of high-order academic open-source solvers exist and are briefly summarised in Table 1, where we include their main capabilities. A common feature in all the solvers selected is that they can vary the order of the method at run time (no need to modify the code). We provide references for each solver and outline here the capabilities of interest when compared to HORSES3D. This list is a non-exhaustive summary where we have not included commercial software. The table reflects that HORSES3D provides a unique set of capabilities to simulate complex flows.

3. Numerical methods

HORSES3D is a Fortran 2008 object-oriented solver, originally developed to solve the compressible Navier-Stokes equations using DG discretisations and explicit time marching. In this section, we detail the compressible solver together with the spatial- and temporal-time advancement schemes.

3.1. Meshing and post-processing

The solver supports curvilinear, hexahedral and conforming meshes in GMSH [21], HDF5 [22] and SpecMesh/HOHQMesh [23] format. Linear meshes generated in Gambit, ICEM, or CGNS can be curved and converted to HDF5 using HOPR [24]. If the user prefers to avoid generating body fitted meshes, we provide a mesh-free immersed boundary capability, which is detailed in (Sec. 4.7).

Tecplot and Paraview [25,26] can be used to post-process the results. We have developed a post-processing utility *horses2plt* that allows interpolating to homogeneous nodal points (from Gauss-Legendre or Gauss-Lobatto) and modifying the polynomial order for visualisation purposes. The tool also allows one to extract derived variables, such as vorticity and Q-criterion (if gradients are enabled during the simulation) or Reynolds stresses (if the statistics are enabled during the simulation).

3.2. Compressible Navier-Stokes

The 3D Navier-Stokes equations can be compactly written as

$$\boldsymbol{u}_t + \nabla \cdot \overleftrightarrow{\mathbf{F}}_e = \nabla \cdot \overleftrightarrow{\mathbf{F}}_v, \tag{1}$$

where $\boldsymbol{u}$ is the state vector of conservative variables $\boldsymbol{u} = [\rho, \rho v_1, \rho v_2, \rho v_3, \rho e]^T$, $\overleftrightarrow{\mathbf{F}}_e$ are the inviscid, or Euler fluxes,

$$\overleftrightarrow{\mathbf{F}}_e = \begin{bmatrix} \rho v_1 & \rho v_2 & \rho u_3 \\ \rho v_1^2 + p & \rho v_1 v_2 & \rho v_1 v_3 \\ \rho v_1 v_2 & \rho v_2^2 + p & \rho v_2 v_3 \\ \rho v_1 v_3 & \rho v_2 v_3 & \rho v_3^2 + p \\ \rho v_1 H & \rho v_2 H & \rho v_3 H \end{bmatrix}, \tag{2}$$

where ρ, e, $H = E + p/\rho$, and p are the density, total energy, total enthalpy and pressure, respectively, and $\vec{v} = [v_1, v_2, v_3]^T$ is the velocity. Additionally, $\overleftrightarrow{\mathbf{F}}_v$ defines the viscous fluxes,

$$\overleftrightarrow{\mathbf{F}}_v = \begin{bmatrix} 0 & 0 & 0 \\ \tau_{xx} & \tau_{xy} & \tau_{xz} \\ \tau_{yx} & \tau_{yy} & \tau_{yz} \\ \tau_{zx} & \tau_{zy} & \tau_{zz} \\ \sum_{j=1}^{3} v_j \tau_{1j} + \kappa T_x & \sum_{j=1}^{3} v_j \tau_{2j} + \kappa T_y & \sum_{j=1}^{3} v_j \tau_{3j} + \kappa T_z \end{bmatrix}, \tag{3}$$

Table 1
Summary of high-order solvers and capabilities.

	HORSES3D	Nek5000	Nektar++	Semtex	deal.II	Flexi/Fluxo	Trixi.jl	PyFR
References		[11]	[12,13]	[14]	[15]	[16,17]	[18,19]	[20]
Language	Fortran	Fortran	C++	C++	C++	Fortran	Julia	Python
Spatial discretisation	DG	CG	CG/DG	CG	DG	DG	DG	FR
Incompressible	✓	✓	✓	✓	✓	✗	✗	✓
Compressible	✓	✗	✓	✗	✓	✓	✓	✓
Energy/Entropy stable	✓	✗	✗	✗	✗	✓	✓	✗
p-adaptation	✓	✗	✓	✗	✓	✓	✗	✓
Steady/Unsteady	S/U	U	S/U	U	S/U	U	U	S/U
LES	✓	✓	✗	✓	✗	✓	✗	✓
RANS	✓	✓	✗	✗	✗	✓	✗	✗
Multiphase	✓	✗	✗	✗	✓	✗	✗	✓
Shocks	✓	✗	✓	✗	✓	✓	✓	✓
Particles	✓	✓	✗	✓	✓	✓	✗	✗
Aeroacoustics	✓	✗	✓	✗	✓	✓	✓	✗
Immersed Boundaries	✓	✗	✓	✗	✓	✗	✗	✗
Parallelisation								
Shared/Distributed - Memory	S/D	D	D	D	S/D	D	S/D+GPU	S/D+GPU

where κ is the thermal conductivity, T_x, T_y and T_z denote the temperature gradients and the stress tensor $\boldsymbol{\tau}$ is defined as $\boldsymbol{\tau} = \mu(\nabla\vec{v} + (\nabla\vec{v})^T) - 2/3\mu\boldsymbol{I}\nabla\cdot\vec{v}$, with μ the dynamic viscosity and $\boldsymbol{I}$ the three-dimensional identity matrix.

3.3. Spatial discretisation: discontinuous Galerkin

HORSES3D discretises the Navier-Stokes equations using the Discontinuous Galerkin Spectral Element Method (DGSEM), which is a particularly efficient nodal version of DG schemes [27]. For simplicity, here we only introduce the fundamental concepts of DG discretisations. More details can be found in [28,29].

The physical domain is tessellated with non-overlapping curvilinear hexahedral elements, e, which are geometrically transformed to a reference element, el. This transformation is performed using a polynomial transfinite mapping that relates the physical coordinates $\vec{x}$ and the local reference coordinates $\vec{\xi}$. The transformation is applied to (1) resulting in the following:

$$J\boldsymbol{u}_t + \nabla_\xi \cdot \overleftrightarrow{\tilde{\mathbf{F}}}_e = \nabla_\xi \cdot \overleftrightarrow{\tilde{\mathbf{F}}}_v, \tag{4}$$

where J is the Jacobian of the transfinite mapping, ∇_ξ is the differential operator in the reference space and $\overleftrightarrow{\tilde{\mathbf{F}}}$ are the contravariant fluxes [27].

To derive DG schemes, we multiply (4) by a locally smooth test function ϕ_j, for $0 \le j \le P$, where P is the polynomial degree, and integrate over an element el to obtain the weak form

$$\int_{el} J\boldsymbol{u}_t\phi_j + \int_{el} \nabla_\xi \cdot \overleftrightarrow{\tilde{\mathbf{F}}}_e\phi_j = \int_{el} \nabla_\xi \cdot \overleftrightarrow{\tilde{\mathbf{F}}}_v\phi_j. \tag{5}$$

We can now integrate by parts the term with the inviscid fluxes, $\overleftrightarrow{\tilde{\mathbf{F}}}_e$, to obtain a local weak form of the equations (one per mesh element) with the boundary fluxes separated from the interior

$$\int_{el} J\boldsymbol{u}_t\phi_j + \int_{\partial el} \overleftrightarrow{\tilde{\mathbf{F}}}_e \cdot \hat{\mathbf{n}}\phi_j - \int_{el} \overleftrightarrow{\tilde{\mathbf{F}}}_e \cdot \nabla_\xi\phi_j = \int_{el} \nabla_\xi \cdot \overleftrightarrow{\tilde{\mathbf{F}}}_v\phi_j, \tag{6}$$

where $\hat{n}$ is the unit outward vector of each face of the reference element ∂el. We replace discontinuous fluxes at inter–element faces by a numerical inviscid flux, $\overleftrightarrow{\tilde{\mathbf{F}}}^\star_e$, to couple the elements,

$$\int_{el} J\boldsymbol{u}_t \cdot \phi_j + \int_{\partial el} \overleftrightarrow{\tilde{\mathbf{F}}}^\star_e \cdot \hat{\mathbf{n}}\phi_j - \int_{el} \overleftrightarrow{\tilde{\mathbf{F}}}_e \cdot \nabla_\xi\phi_j = \int_{el} \nabla_\xi \cdot \overleftrightarrow{\tilde{\mathbf{F}}}_v\phi_j. \tag{7}$$

This set of equations for each element is coupled through the Riemann fluxes $\overleftrightarrow{\tilde{\mathbf{F}}}^\star_e$, which governs the numerical characteristics, see for example the classic book by Toro [30]. Note that one can proceed similarly and integrate the viscous terms by parts (see, for example, [31–34]). The viscous terms require further manipulations to obtain usable discretisations (Bassi Rebay 1 and 2 or Interior Penalty) and, for the sake of simplicity, here we retain the simple volume form:

$$\int_{el} J\boldsymbol{u}_t \cdot \phi_j + \int_{\partial el} \underbrace{\overleftrightarrow{\tilde{\mathbf{F}}}^\star_e \cdot \hat{\mathbf{n}}}_{\text{Convective fluxes}} \phi_j - \int_{el} \overleftrightarrow{\tilde{\mathbf{F}}}_e \cdot \nabla_\xi\phi_j = \int_{el} \Big(\underbrace{\nabla_\xi \cdot \overleftrightarrow{\tilde{\mathbf{F}}}_v}_{\text{Viscous term}} \Big) \cdot \phi_j. \tag{8}$$

The final step, to obtain a usable numerical scheme, is to approximate the numerical solution and fluxes by polynomials (of order P) and to use Gaussian quadrature rules to numerically approximate volume and surface integrals. In HORSES3D we allow for Gauss-Legendre or Gauss–Lobatto quadrature points. The former are more accurate but the latter allow for the derivation of energy/entropy stable formulations, see details in (Sec. 3.5)

> In HORSES3D we have implemented the following Riemann fluxes: Central, Lax-Friedrichs, Rusanov, Roe, Low dissipation Roe, Roe-Pike, matrix dissipation; and the viscous fluxes: Bassi-Rebay 1 and 2 and Interior Penalty (symmetric version).

Riemann solvers are the classic option to include numerical dissipation in DG schemes [35,1], however, for large enough Reynolds numbers, we typically include turbulence closure models for additional dissipation (Sec. 4.1).

3.4. Implementation of alternate physics

The code was initially developed to solve the Navier-Stokes equations (1) in the form of a conservation law. Most of the physical systems introduced in Sec. 4 can be described within this framework with alternate definitions of the conserved variables $\boldsymbol{u}$, convective fluxes $\overleftrightarrow{\mathbf{F}}_e$ and viscous fluxes $\overleftrightarrow{\mathbf{F}}_v$. Of course, changing the definitions of these quantities affects the numerical part of the solver, e.g., the convective Riemann fluxes used. Among the exceptions, we have the multiphase system described in Sec. 4.3, which

contains non-conservative terms that cannot be described within this framework, see details in Sec. 4.3.

All the code that includes the definition of conserved variables, fluxes, and the numerics associated (e.g., numerical fluxes) is organised into a library called *Physics*, so the set of equations that the code solves can be changed by modifying the Physics library. The alternate systems described in Sec. 4 have been constructed in this way. At compilation time, different executables are generated (one for each different physics implemented in HORSES3D). This framework allows for some flexibility, while keeping the efficiency of a one-physics-tailored code.

3.4.1. The problem file

HORSES3D incorporates a dynamic library that is compiled separately (after) from the main code compilation. This library includes several useful subroutines that enhance the interaction level of the user with the code and provides interfaces so that the core solver does not have to be altered for special cases. The main subroutines included are:

- UserDefinedStartup: a subroutine that is called before any other routines in the code.
- UserDefinedFinalSetup: a subroutine that is called after the mesh is read to allow mesh-related initialisations or memory allocations.
- UserDefinedInitialCondition: a subroutine that is called to set the initial condition for the flow.
- UserDefinedState/Grad/Neumann: a set of subroutines that allows the definition of user-defined boundary conditions.
- UserDefinedPeriodicOperation: Called before/during/after/periodically at every time-step to allow periodic operations to be performed.
- UserDefinedSourceTermNS: Called to apply source terms to the system of equations.
- UserDefinedFinalize: Called after the solution is computed to allow, for example, error tests to be performed.
- UserDefinedTermination: Called at the end of the main driver after everything else is done.

The problem file library allows the user to customise the behaviour of the solver for specific purposes. For instance, a pressure probe could be defined using specific interpolation operators to extract the state at a specific location in the flow using UserDefinedFinalSetup. The necessary memory for this variable can be allocated in UserDefinedStartup, and post-processes (e.g., filtered) through UserDefinedPeriodicOperation. The final result could then be output in the desired format using UserDefinedFinalize.

3.5. Energy/entropy stable versions

HORSES3D can use Gauss-Lobatto (GL) quadrature points (in lieu of Gauss) for the integral approximation, which, through its Summation-By-Parts Simultaneous-Approximation Term (SBP-SAT) property [36,37], allows one to recover the continuous property of energy/entropy conservation at a discrete level. This enhances the robustness of the simulations, especially in the presence of high numerical aliasing (e.g., under-resolved turbulence) [38].

There is a plethora of work focused on the construction of entropy- and energy-stable schemes for the discontinuous Galerkin method [36,39–41,28,42–44,47–54] and the references therein. Useful reviews on energy/entropy stable schemes for discontinuous Galerkin schemes are [40,45,46]. HORSES3D also implements the split form approximation, as described in the references. Although similar to what is described in Sec. 3.3, the split form uses two-point fluxes and has better stability properties, albeit at higher cost.

The following energy/entropy stable discretisations are available for the compressible Navier-Stokes and RANS solvers in HORSES3D: Morinishi [47,40], Ducros [48,40], Kennedy-Gruber [49], Pirozzoli [50], Chandrasekar [51,52]. Two entropy stable discretisations for the incompressible Navier–Stokes solver are available based on the splitings presented in [53]. For the Multiphase flow solver a skew symmetric form based on [54] is available.

Recently, we have extended entropy-stable schemes to the Spalart-Allmaras RANS equations [55] and multiphase flows (i.e. Navier-Stokes coupled to Cahn-Hilliard with local $p-adaptation$ [56]), and these have been implemented in HORSES3D.

3.6. Spatial p-adaptation

The code allows the selection of a different polynomial order (P) for each element. Additionally, the polynomial order can be different in each spatial dimension (P_x, P_y, P_z), allowing anisotropic p-adaptation. Treatment of the geometrical representation of the p-non-conforming faces for general curvilinear hexahedral elements is performed following the rules presented in [57]. The coupling between the faces of elements with different polynomial orders is performed using the mortar method [58], which does not retain entropy stability. Selection of the polynomial order inside each element can be performed before the simulation starts (along with the mesh generation process) or as an automated process for steady or unsteady simulations. The automatic adaptation process requires a sensor that specifies which elements should be refined/coarsened. Details of sensor computation and adaptation process are explained in the following sections.

3.6.1. Sensors for adaptation

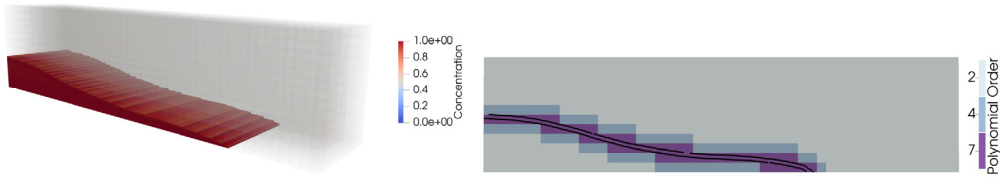
Our preferred adaptation algorithm included in HORSES3D uses the truncation error, which is an efficient sensor for mesh adaptation [59,60]. Like adjoint mesh adaptation, truncation error adaptation targets the source of the discretisation error, rather than adapting polluted regions [59]. Additionally, we have shown that the truncation error controls all functional errors (e.g., error in lift or drag prediction) [3].

The truncation error can be estimated using the τ-estimation technique. The estimation of the truncation error is cheap compared to adjoint techniques. The foundation of the $\tau-$ estimation for high-order was established in [61,62]. This technique permits the estimation of the truncation error on a hierarchy of meshes and therefore allows the selection (in one step) of the correct polynomial order for each element in each spatial direction. τ-estimation was recently improved [5] to be a byproduct of a multigrid cycle.

Automatic mesh adaptation can also be performed using a feature-based approach. This is applied to perform p-adaptation for the Cahn-Hilliard equation and for the incompressible Navier-Stokes/Cahn-Hilliard system (multiphase flows), see Sec. 4.2 and Sec. 4.3. The application of interest is to locally refine the region of the interface between the different phases, as it contains large gradients that need to be resolved to ensure the correctness of the solution.

3.6.2. Adaptation process

The truncation error adaptation algorithm performs mesh p-adaptation for steady and unsteady simulations. For steady flows, a simulation is conducted with a moderate polynomial order, P,



(a) $t = 2$ (b) $t = 2$

Fig. 2. Instances of the dam break test case showing the initial condition and the flow configuration at $t = 2\,s$. Taken from [56].

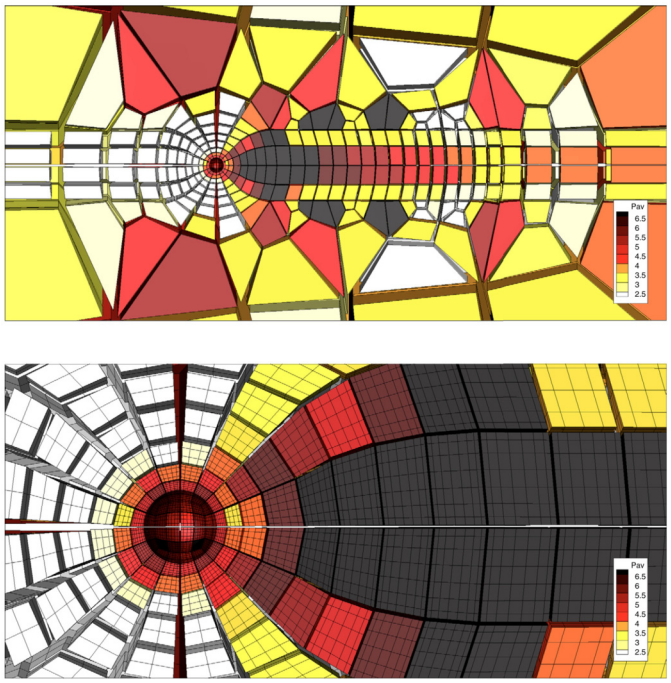


Fig. 1. Example of a p-adapted mesh for the flow around a sphere at Reynolds 200. The contours indicate the average polynomial order ($Pav = (P_x + P_y + P_z)/3$). Bottom figure shows zoomed view near the sphere wake. Taken from [6].

and converged until the residual is smaller than the truncation error threshold set as the objective for the adapted mesh. Then, the truncation error is estimated with the τ-estimation technique for all the polynomial orders coarser than the initial one. For every element, the lowest polynomial order (or polynomial order combination for anisotropic mesh adaptation) that meets the truncation error threshold is selected. If the polynomial orders from one to P do not meet the truncation error threshold, an extrapolation process is performed to select the correct polynomial order. In Fig. 1 an example of a p-adapted mesh based on truncation errors is shown. See [3,4,6] for more examples of truncation error-based adaptation for the compressible Navier-Stokes equations. For unsteady flows, the process is similar; however, no initial converged simulation is required, the truncation error is estimated and the mesh is adapted every prescribed number of iterations [56].

The adaptation process requires the user to select a target truncation error threshold for the adapted mesh. This threshold does not have any direct physical or engineering meaning and has been seen as a drawback of the methodology in the past. However, we have recently shown [63] how to link the truncation error threshold with any flow functional (e.g., lift, drag), bypassing this limitation.

The multiphase flow applications (Sec. 4.3) are of unsteady nature, and thus require the use of dynamic adaptation. In this case, it is crucial to have an adequate resolution in the region of the interface between different fluids. The tracking algorithm uses the value of the phase field parameter ϕ of the Cahn-Hilliard equation. The interface is defined as the region where $0.1 \leq \phi \leq 0.9$. Thus, knowing the location of the interface throughout the simulation, we adapt the elements that contain part of the interface between different phases. This process, shown in Fig. 2, is detailed in [64,56], as well as the method through which it can be automated.

3.7. Time advancement

There are several temporal integration strategies in HORSES3D, which are tailored for specific solvers. By default, the low-storage explicit Runge-Kutta of order three [65] is used as a time-stepping strategy for any given problem. This can be changed by the user to an explicit Runge-Kutta 4^{th} order [66] or to more sophisticated methods described below.

For steady-state simulations, the non-linear p-multigrid method (also known as Full Approximate Scheme – FAS) [6] is employed. FAS employs several explicit (local time-stepping, optimised Runge-Kutta coefficients for steady-state) [67,68] and semi-implicit (BIRK, LU-SGS, ILU(0)) [69–71] convergence acceleration strategies, presented in Fig. 4. An example of convergence for a laminar test case is presented in Fig. 3.

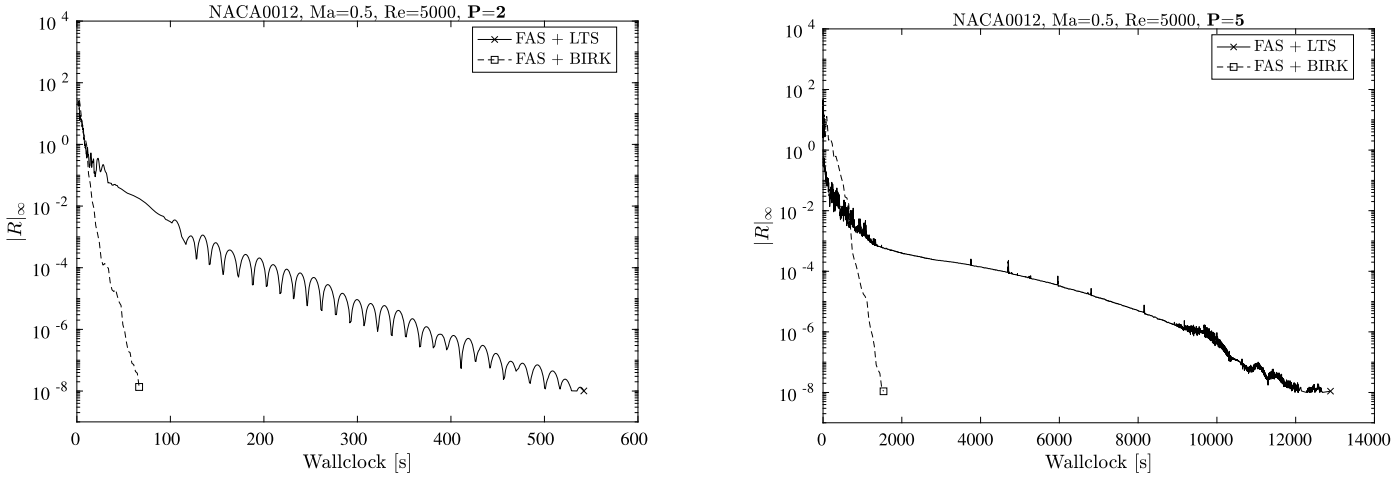
For time-accurate simulations (e.g., Large Eddy Simulations), implicit time integration can be used with Backward Difference Formulas (BDF) of order $1, \ldots, 5$, or 6^{th} order Rosenbrock-type Runge-Kutta schemes [72]. The Jacobian can be computed analytically for the Navier-Stokes equations and also numerically when other equations need to be included (e.g., Spalart-Allmaras) using a colouring algorithm [73,74] to improve performance.

3.8. Parallelisation and efficiency

To reduce the computational cost, all techniques are parallelised for hybrid systems to run on multiprocessor CPUs combining MPI and OpenMP. All the multiphysic implementations have been parallelised using OPENMP (shared memory). In addition, MPI (distributed memory) parallelisation is available for all physics, except the multiphase and particles solvers. Numerical experiments for explicit time-stepping have been performed on the Taylor-Green Vortex (TGV) problem [75]. The TGV problem has been widely used to report the subgrid-scale modelling capabilities of ILES approaches and discretisations [43,76,77]. The results are shown in Fig. 5.

The TGV case is simulated in a 64^3 mesh with polynomial orders of three to six, giving approximately 16 to 90 million of degrees of freedom. We use Gauss–Lobatto interpolation points and Pirozzoli's split–form, with Roe (convective) and BR1 (diffusive) fluxes. The number of total cores ranges from one to 700, and the number of OpenMP threads when enabled is 48. Simulations were performed in the MareNostrum-3 supercomputer [78].

We observe very good scalability when using high polynomial orders and MPI + OpenMP architectures up to a thousand cores; see Fig. 5.b, where we get 86% of the ideal performance for 600 cores and $P = 6$.



(a) Result for NACA0012 test case with 2^{nd} order ($P = 1$) discretisation.

(b) Result for NACA0012 test case with 5^{th} order ($P = 4$)discretisation.

Fig. 3. Figures present convergence for a laminar test case (NACA0012 airfoil with Re = 5000, Ma = 0.5 and AoA = 0°) using FAS with explicit and semi-implicit convergence acceleration. The explicit FAS uses a five stage 2^{nd} order Runge-Kutta smoother [68] with local time-stepping. The semi-implicit FAS uses the BIRK smoother [71]. A low (left figure) and high (right figure) uniform polynomial discretisations are considered.

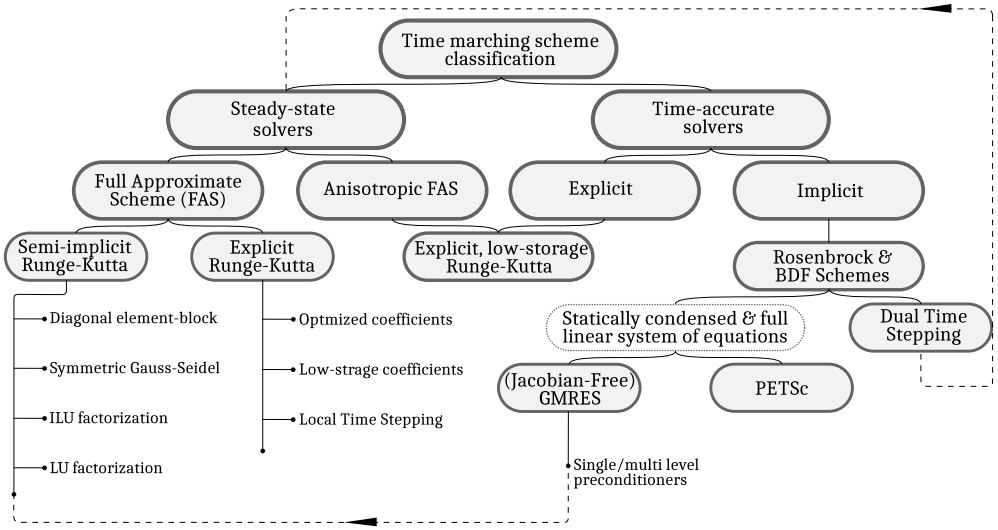


Fig. 4. Classification of time-marching schemes employed in HORSES3D.

For time-resolved LES simulations, we typically use explicit schemes (RK3 or RK4), which have the advantage of parallelising very efficiently (see Fig. 5). Implicit solvers are very useful to converge to steady states (laminar or RANS), particularly when applying p-multigrid with local time stepping or p-multigrid with BIRK [79,71,72]. In HORSES3D, we have also implemented a dual time stepping technique to take advantage of the efficient steady state solvers (p-multigrid) for unsteady flows. The latter option is cost effective when small features can be sacrificed by using CFL numbers of the order of 100-1000. Purely implicit high order solvers (e.g., block-Jacobi preconditioned GMRES) are not easy to converge for high order schemes, in our experience.

4. Multiphysics

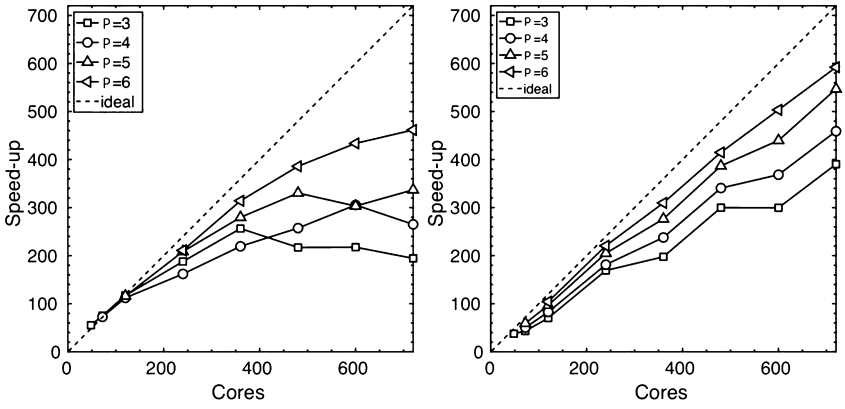
In this section, we detail the various multiphysics applications included in HORSES3D. Namely, we detail the turbulence closure models (RANS and LES), the incompressible flow formulation, multiphase modelling, shock capturing, flow-particle dynamics, aeroacoustics and immersed boundaries.

4.1. Turbulence closure models

A particular challenge related to the numerical simulation of fluid flows relates to turbulence modelling. The main characteristic of turbulent flows is the presence of a broad spectrum, where a wide range of length and time scales coexist [80]. It is the multiscale character (in time and space) of turbulent flows, which scales with the Reynolds number (i.e. ratio of inertia to viscous forces) that renders the numerical treatment of turbulence a difficult task.

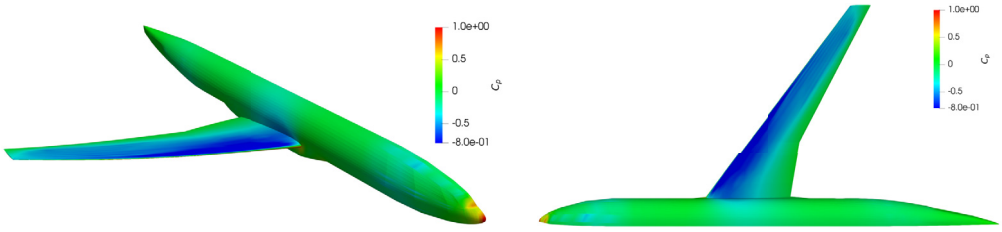
The Direct Numerical Simulation (DNS) approach solves the Navier-Stokes equations without additional modelling. Consequently, it requires very high spatial and temporal resolution and is thus restricted to moderately low Reynolds numbers. To compute large Reynolds number flows in complex configurations, it is necessary to introduce a certain degree of modelling to limit the cost. There are two main approaches: the Reynolds Averaged NS (RANS) approach and the Large Eddy Simulation (LES) technique. Both types are included in HORSES3D.

The RANS solver uses the negative Spalart-Allmaras model from [81,67], while the LES models consider both explicit or implicit



(a) MPI parallelization (b) Hybrid MPI+OpenMP parallelization

Fig. 5. HORSES3D scalability solving the TGV problem with a 64^3 mesh using a) MPI and b) MPI/OpenMP.



(a) Isometric view. (b) Top view.

Fig. 6. Pressure contour over the surface of the Common Research Model (CRM) for M=0.85, Re=$5 \cdot 10^6$ and AoA = 2.3° using the HORSES3D RANS module with split-form inviscid fluxes. The computations have been done on a 3rd order ($P = 2$) hexahedral mesh from [83].

subgrid modelling. An example of the results derived using the HORSES3D framework is presented in Fig. 6, where we consider the Common Research Model [82].

The explicit LES approach uses spatial filtering where the large structures are resolved, reducing the modelling to the small unresolved turbulent structures (i.e. small eddies), which behave in an isotropic fashion. We have implementations for classic LES models, including Smagorinsky [84,85], Wale [86], and Vreman [87]. In addition, we developed in [77] a SVV-Smagorinsky model where the Spectral Vanishing Viscosity technique was combined with a Smagorinsky LES model to enhance the closure model. The model was capable of maintaining low dissipation levels for laminar flows, while providing the correct dissipation for all wave–number ranges in turbulent regimes.

In recent years, Implicit Large Eddy Simulation (ILES) methods [88] have gained popularity. This approach, increasingly popular when combined with high-order numerical methods (e.g., DG), uses the dissipation inherited from the numerical scheme to mimic turbulent effects of unresolved scales. DG methods have dissipation and dispersive errors, which are confined to high wavenumbers, and therefore introduce numerical dissipation primarily to under-resolved wave-lengths [89,90]. This property makes them very suitable for ILES, since dissipation only acts at under-resolved scales, as explained in [29,91–93].

Riemman solvers are the classic option to include numerical dissipation in DG schemes [1,35], since they naturally arise when discretising the non-linear terms. A comparison of different fluxes for homogeneous turbulence can be found in [94,77]. A different option is to modify the viscous terms to enhance the approximation's dissipative properties. The modification has been proposed in [29] using an increased penalty parameter (compared to the minimum required to ensure coercivity of the scheme) when discretising the viscous terms using an interior penalty formulation.

When combined with high-order DG methods, ILES provide valuable results even for challenging problems such as the prediction of turbulent quantities, detached flows and laminar-turbulent transition on airfoils, see, for example, [95,29,96,88,77,97,98]. Finally, a promising approach is to combine robust energy-stable methods (e.g., skew symmetric variants) with ILES techniques (e.g., Spectral Vanishing Viscosity) to provide accurate simulation at high Reynolds numbers, see [91,77].

As an example of ILES performance, we calculated the averaged values for the lift and drag on a NACA0012, using a hexahedral mesh with 18000 elements, with $P = 4$ (2.2 million degrees of freedom). Fig. 7 compares the aerodynamic coefficients with the experimental data for various angles of attack and the two solvers. Fig. 7 [Left] shows the lift coefficient against AoA and Fig. 7 [Right] depicts the lift-drag polar for $\mathrm{Re} = 1 \times 10^6$. We observe very good agreement with the experimental data.

> The following turbulence models are available in HORSES3D: RANS: Spalart-Allmaras; explicit LES: Smagorinsky, Wale, Vreman, SVV-Smagorinsky; implicit LES: energy/entropy stable formulations and dissipative fluxes.

4.2. Incompressible flows

From among the various incompressible Navier-Stokes models, HORSES3D uses an artificial compressibility formulation [99], which converts the elliptic problem into a hyperbolic system of equations, at the expense of a non divergence–free velocity field. However, it allows one to avoid constructing an approximation that satisfies the inf–sup condition, see [100,101] and the refer-

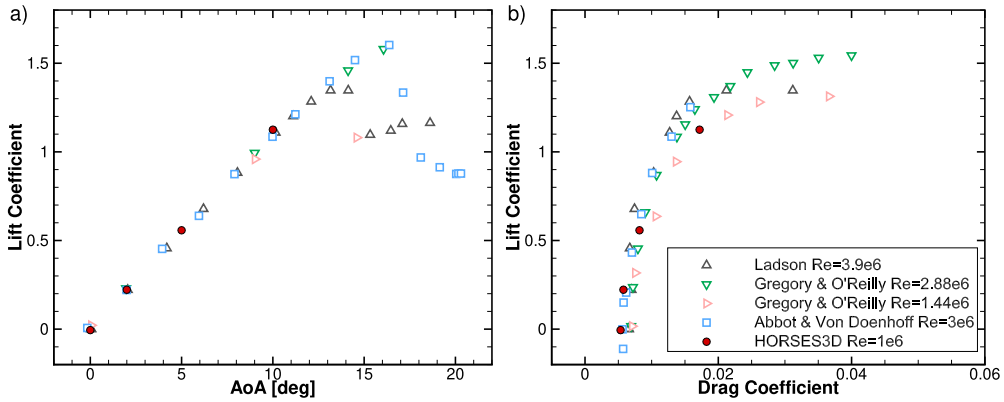


Fig. 7. Lift and drag for a NACA 0012 at Re=1e6; comparisons between HORSES3D (ILES version) with experimental data. Adapted from [96].

ences therein. The artificial compressibility model is commonly combined with dual time–stepping, which solves an inner pseudo-time-step loop until velocity divergence errors are lower than a selected threshold, to then perform the physical time advancement. However, we have found [53] that solving the incompressible Navier-Stokes equations with artificial compressibility (without the pseudo-time-step loop) can achieve satisfactory results even in transient simulations. This methodology is well suited for use as a fluid flow engine for interface–tracking multiphase flow models, as it allows the density to vary spatially $\rho\left(\vec{x},t\right)$.

The artificial compressibility system of equations is

$$\rho_t + \vec{\nabla}\cdot\left(\rho\vec{u}\right) = 0, \tag{9}$$

$$\left(\rho\vec{u}\right)_t + \vec{\nabla}\cdot\left(\rho\vec{u}\vec{u}\right) = -\vec{\nabla}p + \vec{\nabla}\cdot\left(\frac{1}{\mathrm{Re}}\left(\vec{\nabla}\vec{u}+\vec{\nabla}\vec{u}^{T}\right)\right) + \frac{1}{\mathrm{Fr}^2}\rho\vec{e}_g, \tag{10}$$

$$p_t + \rho_0 c_0^2\vec{\nabla}\cdot\vec{u} = 0, \tag{11}$$

where Re is the Reynolds number, Fr is the Froude number, c_0^2 is the artificial speed of sound, and $\vec{e}_g$ is the gravitational acceleration. The impact of the artificial speed of sound on the accuracy in steady–state problems is not significant [53]. For transient simulations, however, the choice of the artificial speed of sound is important. The typical values used in HORSES3D are $\rho_0 c_0^2 \in [10^3, 10^4]$ so that pressure waves remain low, while not inducing severe time–step restrictions for explicit time integrators [53].

4.3. Multiphase

Phase field models describe the phase separation dynamics of two immiscible liquids by minimising a chosen free–energy. For an arbitrary free–energy function, it is possible to construct different phase field models. Among the most popular, one can find the Cahn–Hilliard [102] and the Allen–Cahn [103] models. The popularity of the first, despite being a fourth-order operator in space, comes from its ability to conserve the phases. HORSES3D permits the solution of the Cahn-Hilliard equation. For a detailed explanation of the model, see [104,105].

The multiphase flow solver implemented in HORSES3D is constructed by a combination of the diffuse interface model of Cahn–Hilliard [102] with the incompressible Navier–Stokes equations with variable density and artificial compressibility [99]. This model is entropy stable and guarantees phase conservation with an accurate representation of surface tension effects. For a detailed explanation of the model, see [28]. The modified entropy-stable version approximates

$$c_t + \vec{\nabla}\cdot\left(c\vec{u}\right) = M_0\vec{\nabla}^2\mu, \tag{12}$$

$$\sqrt{\rho}\left(\sqrt{\rho}\vec{u}\right)_t + \vec{\nabla}\cdot\left(\frac{1}{2}\rho\vec{u}\vec{u}\right) + \frac{1}{2}\rho\vec{u}\cdot\vec{\nabla}\vec{u} + c\vec{\nabla}\mu = -\vec{\nabla}p + \vec{\nabla}\cdot\left(\eta\left(\vec{\nabla}\vec{u}+\vec{\nabla}\vec{u}^{T}\right)\right) + \rho\vec{g}, \tag{13}$$

$$p_t + \rho_0 c_0^2\vec{\nabla}\cdot\vec{u} = 0, \tag{14}$$

where c is the phase field parameter, M_0 is the mobility, μ is the chemical potential, η is the viscosity and c_0 is the artificial speed of sound. In [28], we provide a comparison between standard and entropy-stable discretisations. Through several numerical experiments, the superior robustness characteristics of the entropy-stable scheme are highlighted in comparison to the standard scheme. Among the test cases considered, there is a two–phase flow of oil and water within a pipe [28] and a 3D dam break test case [56].

For phase field modelling, it is important to adequately resolve the region of the interface between different phases, as it contains large gradients. In HORSES3D, the local refinement around the interface is carried out through p-adaptation. The original scheme presented in [28] has been extended to support p-non-conforming elements and has been modified accordingly so that it is entropy-stable even for p-non-conforming elements, see Fig. 8. The detailed analysis and modifications to the original scheme are presented in [56].

4.4. Shock capturing

Supersonic flows present additional difficulties to high-order spectral elements, where the favourable accuracy and convergence properties of these schemes can be damaged by the Gibbs phenomenon near shocks. HORSES3D implements a special type of artificial viscosity to handle shocks. In the case of the Navier-Stokes equations, discontinuities due to compressibility effects have an actual width proportional to the viscosity, μ [106]. This motivates the introduction of an artificial viscous term $\overleftrightarrow{\mathbf{F}}_a$ into the equations to control the effective μ at each element to smooth the solution.

HORSES3D includes two options for $\overleftrightarrow{\mathbf{F}}_a$: the viscous flux of the Navier-Stokes equations and the Guermond-Popov entropy-stable flux [107], which also includes a regularisation term in the density equation. Regardless of the selected viscous flux, and to minimise artificial viscosity in smooth regions, we introduce an Spectral Vanishing Viscosity (SVV) formulation and apply a frequency-dependent filter kernel $\mathcal{F}$ to these fluxes, so that the additional term has the form $\overleftrightarrow{\mathbf{F}}_a^{\mathcal{F}} = \mathcal{F}\star\overleftrightarrow{\mathbf{F}}_a$. We implement the filter kernels of [108–110]. Since the code is a nodal DGSEM framework, we transform the fluxes to the Legendre modal space, $\{L_i\}$, where the application of this filter is simply an element-wise product of the fluxes and the kernel, and then we return to the nodal space to recover the filtered flux.

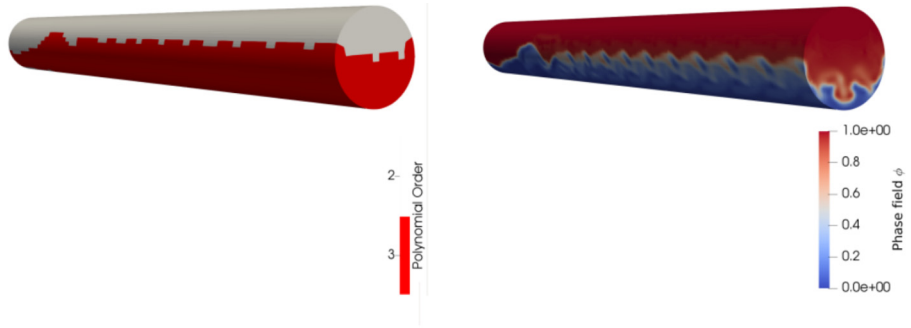


Fig. 8. Annular flow test case of water and oil. On the left, the polynomial order distribution is presented for the p-adaptive scheme based on the interface tracking refinement. On the right, the flow structure is presented for the developed flow. Taken from [56].

Fig. 9. Facing forward step case at $\tau = 10$ with an incoming flow at Ma=3 and 3653 elements with polynomial degree $P = 7$. Adapted from [113].

The stability of this new term is analysed in terms of the evolution of the entropy. Physically plausible solutions of the Navier-Stokes equations must agree with elemental laws such as the conservation of the energy or a non-decreasing entropy. We impose this by defining a mathematical entropy function that represents the underlying law and requiring it to be bounded. In this case, the mathematical entropy, S, is defined in terms of the non-decreasing physical entropy as

$$S = -\rho s, \quad s = \ln p - \gamma \ln \rho. \tag{15}$$

We also introduce the concept of entropy variables $\boldsymbol{w}$, which are computed from (15) as

$$\boldsymbol{w} = \frac{\partial S}{\partial \boldsymbol{u}}, \quad \boldsymbol{u} = (\rho, \rho\vec{v}, \rho e)^T. \tag{16}$$

To ensure the stability of the new term, a detailed semi-discrete entropy analysis (see [111,112,55]) shows that filtering has to be applied in a very specific form. Expressing the baseline flux as

$$\overleftrightarrow{\mathbf{F}}_a = \mathcal{B}\overleftrightarrow{\mathbf{G}}, \quad \mathcal{B} = \mathcal{L}^T\mathcal{D}\mathcal{L}, \quad \overleftrightarrow{\mathbf{G}} = \nabla \boldsymbol{w}, \tag{17}$$

where $\mathcal{B}$ is a symmetric matrix, and $\mathcal{L}$ and $\mathcal{D}$ are the matrices of its $\mathbf{L}^T\mathbf{D}\mathbf{L}$ decomposition, the filtered flux, $\overleftrightarrow{\mathbf{F}}_a^{\mathcal{F}}$, is obtained by filtering only one half of (17),

$$\overleftrightarrow{\mathbf{F}}_a^{\mathcal{F}} = \frac{1}{\sqrt{J}}\mathcal{L}^T\sqrt{\mathcal{D}}\mathcal{F}\star\left(\sqrt{J\mathcal{D}}\mathcal{L}\overleftrightarrow{\mathbf{G}}\right), \tag{18}$$

where J is the Jacobian of the transformation from the reference element to the physical elements of the mesh, see Sec. 3.3. A detailed explanation of this derivation can be found in [113].

The SVV approach has proven to be useful for the simulation of turbulent and supersonic flows (see Fig. 9); however, the treatment of discontinuities requires a higher amount of dissipation that can be achieved only by adding non-filtered dissipation in troubled regions. Since this is only required near discontinuities, we use a sensor, s_ρ, based on the average value of the density gradient to detect troubled elements and modify the filter there,

$$s_\rho = \int_{el} J|\nabla_\xi \rho|^2. \tag{19}$$

4.5. Particles

HORSES3D includes a two-way coupled Lagrangian solver. The model presented here is based on [114]. This model is developed to simulate particle-laden turbulent flows subject to radiation, which are found in practical applications such as particle-based solar receivers [115]. The Lagrangian solver is currently implemented for the compressible Navier-Stokes system with explicit time integration.

Particles are tracked along their trajectories according to the simplified particle equation of motion, where only contributions from Stokes drag and gravity are retained,

$$\frac{dy_i}{dt} = u_i, \quad \frac{du_i}{dt} = \frac{v_i - u_i}{\tau_p} + g_i, \tag{20}$$

where u_i and y_i are the *ith* components of velocity and position of the particle, respectively. Furthermore, v_i accounts for the continuous velocity of the fluid at the position of the particle. We consider spherical Stokesian particles, so their mass and aerodynamic response time are $m_p = \rho_p \pi D_p^3/6$ and $\tau_p = \rho_p D_p^2/18\mu$, respectively, ρ_p being the particle density and D_p the particle diameter.

Each particle is considered to be subject to a radiative heat flux I_o. The carrier phase is transparent to radiation, whereas the incident radiative flux on each particle is completely absorbed. Because we focus on relatively small volume fractions, the fluid-particle medium is considered to be optically thin. Under these hypotheses, the direction of the radiation is inconsequential, and each particle receives the same radiative heat flux, and its temperature T_p is governed by

$$\frac{d}{dt}(m_p c_{V,p} T_p) = \frac{\pi D_p^2}{4} I_o - \pi D_p^2 h(T_p - T), \tag{21}$$

where $c_{V,p}$ is the specific heat of the particle, which is assumed to be constant with respect to temperature. T_p is the particle tem-

perature and h is the convective heat transfer coefficient, which for a Stokesian particle can be calculated from the Nusselt number $Nu = hD_p/k = 2$.

In practical simulations, integrating the trajectory of every particle is too expensive. Therefore, particles are agglomerated into parcels, each of them accounting for many particles with the same physical properties, position, velocity, and temperature. The evolution of the parcels is tracked with the same set of equations presented for the particles.

The two-way coupling means that fluid flow is modified because of the presence of particles. Therefore, the Navier-Stokes equations (1) are enriched with the following source terms:

$$\boldsymbol{S} = \beta \begin{bmatrix} 0 \\ \sum_{n=1}^{N_p} \frac{m_p}{\tau_p}(u_{1,n} - v_1)\delta(\mathbf{x} - \mathbf{y}_n) \\ \sum_{n=1}^{N_p} \frac{m_p}{\tau_p}(u_{2,n} - v_2)\delta(\mathbf{x} - \mathbf{y}_n) \\ \sum_{n=1}^{N_p} \frac{m_p}{\tau_p}(u_{3,n} - v_3)\delta(\mathbf{x} - \mathbf{y}_n) \\ \sum_{n=1}^{N_p} \pi D_p^2 h(T_{p,n} - T)\delta(\mathbf{x} - \mathbf{y}_n) \end{bmatrix}, \tag{22}$$

where δ is the Dirac delta function, N_p is the number of parcels, β is the number of particles per parcel and $u_{i,n}$, $\mathbf{y}_{i,n}$, $T_{p,n}$ are the velocity, spatial coordinates, and temperature of the parcel *nth*.

The ordinary differential equation (ODE) system (20), (21) that controls the evolution of parcels is integrated in time using a low storage Runge-Kutta 3 method [65]. The Navier-Stokes equations and the particle equations are integrated sequentially, as sketched in Algorithm 1. This simple implementation does not require to iteratively perform the two-way coupling, at the cost of a reduction in the time integration order of accuracy.

Algorithm 1 Time integration of the two-way coupled Lagrangian solver.

```
t ← 0
for k = 1, number of steps do
    FlowState(t + dt) ← TakeStepNS(t,dt,ParticlesState(t))
    ParticlesState(t + dt) ← TakeStepParticles(t,dt,FlowState(t + dt))
    t ← t + dt
end for
```

The properties of the fluid (velocity, viscosity, and temperature) at the position of the parcel are evaluated using the interpolation routines included in HORSES3D. These routines work at the element level, so a routine to find the element to which the parcel belongs in general unstructured meshes is also included. The Dirac delta, $\delta(\mathbf{x} - \mathbf{y}_n)$, which appears in the source term, see (22), can be dealt with in a simple way in the DG setting, since it can be integrated exactly in the weak form.

The coupled-particle model has been used to analyse a particle-laden flow in a channel subject to radiation, which is representative of particle solar radiation systems. Details of the test case can be found in [115]. The results obtained with HORSES3D were obtained with a Cartesian mesh of 4000 elements and a number of particles per parcel $\beta = 160$, see Fig. 10. The results of HORSES3D are compared in Fig. 11 with the one-dimensional and three-dimensional simulations performed in [115]. The results of HORSES3D do not exactly match the three-dimensional simulations of [115] as the effect of preferential concentration at the inlet was not considered. Besides HORSES3D uses parcels (agglomerations of particles). This artificial concentration of particles creates local peaks in temperature, which results in an increase of the average temperature.

4.6. Computational aeroacoustics

Computational aeroacoustics (CAA) can be divided into two categories. The first one, noise generation, is characterised by turbulence and non-linear interactions near solid surfaces. The second category is noise propagation and can be simplified to a linear propagation of the sound waves.

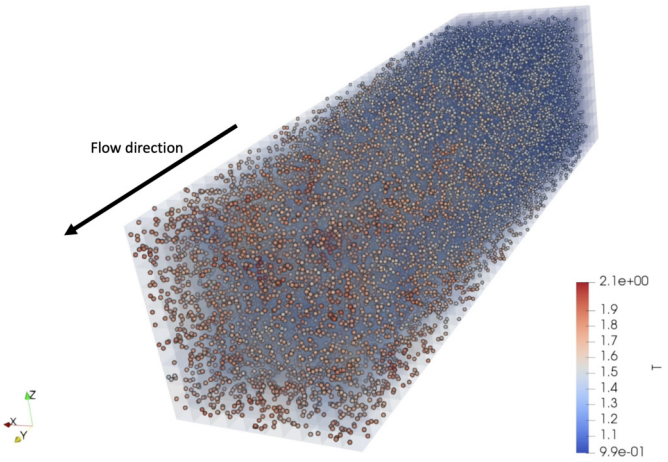


Fig. 10. Visualisation of a particle-laden flow in a channel subject to radiation. Particles and gas are injected into the channel at the same temperature. The temperatures of the particles increase in the direction of the flow as a result of the effect of radiation. Details of the test case are given in [115].

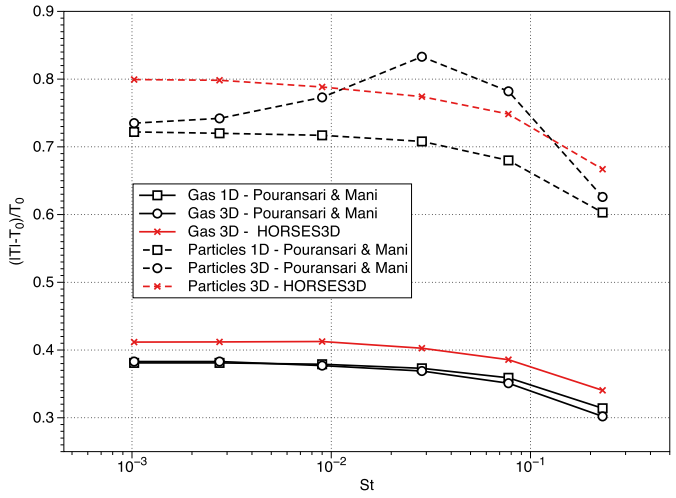


Fig. 11. Mean temperatures of gas and particles at the outlet of a channel subject to radiation. $T_0 = 300$ K is the reference temperature, while St is the Stokes number. Details of the test case are given in [115].

An approach in CAA is to solve the near and far fields together, using the compressible Navier–Stokes equations on the entire computational domain, obtaining all the physical effects without needing propagating models. This approach is called Direct Noise Computation (DNC), and can deal with complex acoustics, such as flow-acoustic feedback. The compressible Navier–Stokes simulations of HORSES3D are spectrally accurate and therefore well suited to simulate small pressure/density fluctuation and related acoustics.

A second possibility for CAA is to use acoustic analogies (hybrid approach), which decouples the near and far fields in two separated computational regions. The near field is computed first and independently of the propagation [116] using the HORSES3D compressible Navier–Stokes solver (with or without LES turbulence models). Subsequently, acoustic sources are extracted from the well-resolved near-field region, to feed the propagation problem, which is solved only in the far field. This decoupling enables faster computations since the near-field that requires highly accurate flow simulations can be performed separately from the acoustic propagation step.

HORSES3D can compute aeroacoustics using the direct approach and also acoustic analogies. In particular, the Ffowcs-Williams and Hawkings (FWH) aeroacoustic analogy [117] is implemented. FWH rearranges the compressible NS equations into an inhomogeneous wave equation in integral form. The FWH uses a fictitious surface located in the near-field, where the Navier–Stokes solution is used to calculate the integral solution. This surface can coincide with the solid boundary of the body (having a solid surface) or can be located at an arbitrary position in the flow field (being a permeable surface). HORSES3D allows both possibilities.

Our implementation follows the formulation by Najafi [118], and Ghorbanias [119]. This formulation clearly separates the flow velocity fluctuations (v_i), surface velocities (v_i), and the free-stream velocity (U_{0i}) as follows:

$$\left(\frac{\partial^2}{\partial t^2}+U_{0i}U_{0j}\frac{\partial^2}{\partial x_i\partial x_j}+2U_{0i}\frac{\partial^2}{\partial x_i\partial t}-c_0^2\frac{\partial^2}{\partial x_i\partial x_i}\right)(H(f)\rho')=$$
$$\left(\frac{\partial}{\partial t}+U_{0i}\frac{\partial}{\partial x_i}\right)\big(Q_i n_i\delta(f)\big)-\frac{\partial}{\partial x_i}\big(L_{ij}n_j\delta(f)\big)$$
$$+\frac{\partial^2}{\partial x_i\partial x_j}\big(T_{ij}H(f)\big), \tag{23}$$

where the zero subscript refers to the undisturbed or free-stream flow, ρ' is the density fluctuation, Q_i, L_{ij}, T_{ij} are the loading, thickness, and quadrupole terms, respectively, n_i is the i^{th} component of the normal at the surface, $H(f)$ is the Heaviside function and $\delta(f)$ is the Dirac delta function. The right-hand side of (23) represents the source terms for the FWH noise model and are calculated from the velocities and pressure (and its time derivative) that are computed by the compressible Navier-Stokes solver. In this implementation, quadrupoles are neglected and so are their contributions to the viscous stress tensor, but they can be easily added when necessary.

The acoustic pressure $p'=p-p_0$ can be obtained from the density fluctuation, as $p'=c_0^2\rho'$ in the far field (where $\rho'/\rho_0<<1$). For wind tunnel scenarios, most of the terms needed to solve (23) cancel out, and as a result, only two components of the acoustic pressure fluctuation are solved, namely

$$p'(\mathbf{x},t)=p'_T(\mathbf{x},t)+p'_L(\mathbf{x},t), \tag{24}$$

$$4\pi\, p'_T=\int_S\left[\frac{\big(1-M_{0i}R_i\big)\dot{Q}_i n_i}{R^*}\right]_{\tau_e}dS-\int_S\left[\frac{U_{0i}\hat{R}_i^*Q_i n_i}{R^{*2}}\right]_{\tau_e}dS, \tag{25}$$

$$4\pi\, p'_L=\int_S\left[\frac{\dot{L}_{ij}n_j\hat{R}_i}{c_0R^*}\right]_{\tau_e}dS+\int_S\left[\frac{L_{ij}n_j\hat{R}_i^*}{R^{*2}}\right]_{\tau_e}dS, \tag{26}$$

where p_T is the thickness pressure fluctuation, p_L is the loading pressure fluctuation, c_0 is the speed of sound, M_0 is the free-stream Mach number, U_0 is the free-stream velocity, R is the phase radius, R^* the amplitude radius (see [120] for a more detailed explanation), and the hat $\hat{()}$ of both R and R^* represents the partial derivatives of the radius quantities. The subscript τ_e indicates that the integrals are calculated at the emission retarded time defined as $\tau_e=t-R/c_0$, see [121] for more details.

For static observers, the retarded time can be calculated analytically, as opposed to when considering bodies in motion. A source time-dominant algorithm is used [122], where at each numerical time-step (also known as *emission* time) the *observer* time is calculated, allowing for a time-advancing approach. All integrals from (25) and (26) are numerically evaluated on each h-mesh face of the FWH surface (solid or permeable), using the high-order (p-mesh) discretisation on the face, which is inherently related to the Gauss-Legendre (or Gauss-Lobatto) nodes.

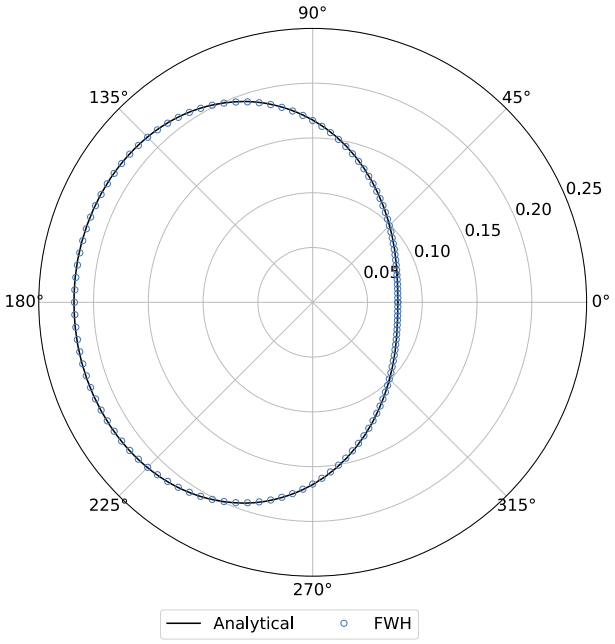


Fig. 12. Stationary monopole in a moving medium [123]. Comparison between analytical and FWH acoustic pressure at $r=20\ell$.

A classic validation case to validate the FWH implementation is the stationary monopole in a moving medium [123]. The monopole tested has a reference length $\ell=1$, an amplitude 1.0, an angular frequency $\omega=4\pi$, a free-stream Mach number $M_0=0.5$, and is centered at the origin $x_0=[0,0,0]$. Fig. 12 shows the prediction of the acoustic pressure rms for observers located at a distance of 20ℓ. Excellent agreement with the analytical solution is obtained.

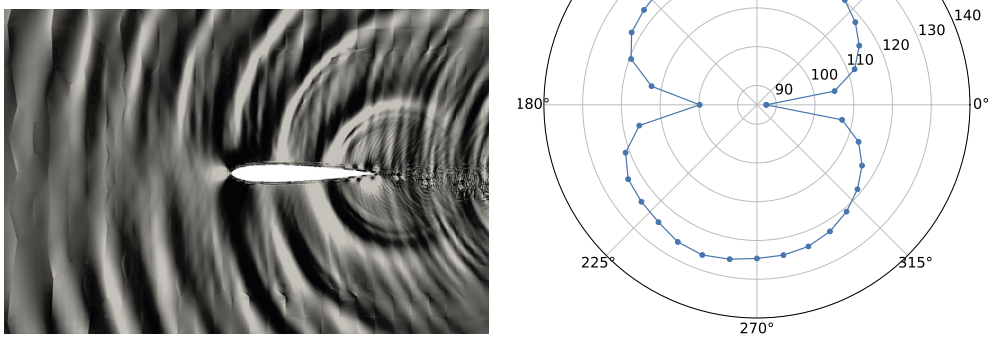
Another case of classical study is the sound generated by an airfoil in an external flow [124,125], where noise is scattered by the trailing edge. We simulate a NACA 0012 at $\mathrm{Re}=1\times10^5$ based on the airfoil chord, and $\mathrm{Ma}=0.4$. DNC computations are performed in the near field, as shown in Fig. 13a for the dilation field, where acoustic waves are shown to be generated at the airfoil trailing edge and propagate upstream to the far field. In addition, Fig. 13b shows the directivity of the acoustic pressure at a $r=2C$, which is computed using the FWH analogy. The expected dipole shape is obtained for this case.

The following two approaches for predicting aeroacoustics are available in HORSES3D: 1-Direct Noise Computations using the compressible solver with or without LES turbulence models; 2-Ffowcs-Williams and Hawkin analogy coupled to the compressible solver with or without LES turbulence models (using a solid or permeable surfaces).

4.7. Mesh free - Immersed Boundary method

In addition to classic body fitted simulations, HORSES3D includes an Immersed Boundary (IB) technique. IB is a mesh-free method that allows the simulation of complex and moving geometries on simple Cartesian meshes. By avoiding the necessity of a body fitting mesh, the grid generation process can be skipped, thus shortening a very time-consuming step.

IB was first introduced by Peskin [126] and comes in many flavours; see the review by Kim and Choi [127]. A possible implementation of IB is volume penalisation, where to simulate the



(a) Dilation field. (b) Far field SPL directivity obtained using FWH.

Fig. 13. NACA 0012 aeroacoustics at $\mathrm{Re} = 1 \times 10^5$.

presence of a geometry, a source term is applied to the Navier-Stokes equations. This approach has recently been extended to high-order methods by our group [128–130]. The method is simple, robust, and can be easily extended to moving geometries.

The IB mask $\chi(\vec{x}, t)$ distinguishes the penalised region (inside the geometry) Ω_s and the outer region (fluid region), Ω_f, as

$$\chi(\vec{x}, t) = \begin{cases} 1, & \text{if } \vec{x} \in \Omega_s \\ 0, & \text{if } \vec{x} \in \Omega_f, \end{cases} \tag{27}$$

where $\vec{x}$ is the coordinate vector. The geometry is considered to be a porous media whose permeability approaches zero to mimic solid geometries. Given the velocity of the geometry $\vec{u}_s = (u_s, v_s, w_s)$, a source term

$$\mathbf{S} = \frac{\chi}{\eta} \begin{pmatrix} 0 \\ \rho u_s - \rho u \\ \rho v_s - \rho v \\ \rho w_s - \rho w \\ \frac{\rho}{2}\left(u_s^2 + v_s^2 + w_s^2\right) - \frac{\rho}{2}\left(u^2 + v^2 + w^2\right) \end{pmatrix} \tag{28}$$

is added to the Navier-Stokes equations, where η is the permeability, ρ and u, v, w the density and velocity components, respectively. Once the mask is generated, the source term (28) is applied to the degrees of freedom and the equations are solved.

In HORSES3D, the mask is constructed using a ray-tracing approach. The input is a CAD (STL) file representing the geometry to be simulated and a generic mesh. A surface-area heuristic KD-tree is built that contains the whole body. This type of KD-tree is known to improve the efficiency of the ray-tracing algorithm. The construction of the KD-tree is parallelised using MPI and OpenMP. The OpenMP construction implements breadth-first and depth-first parallelisation. The first is used at the starting levels of the KD-tree where the number of triangles defining the surface geometry in the STL file is usually big enough to justify the parallelisation of the loops performed on the objects; the latter is used so that each of the remaining KD-tree's branches is performed by a single thread (interested readers are referred to [131]). When MPI is used, the CAD file is split into a number of partitions equal to the number of slaves. A KD-tree is built for each partition, reducing the computational time since the total number of triangles in the partitions is smaller than the initial one.

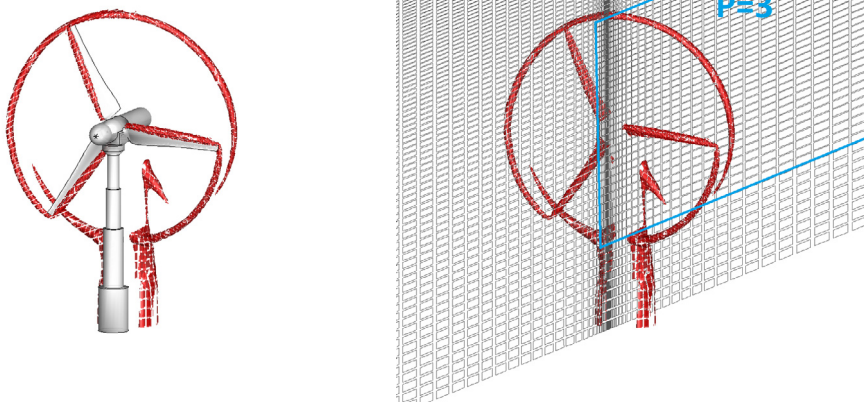
A key aspect of IB is the computation of the aerodynamic forces acting on the body, which requires a reconstruction of the surface data. The idea is to find the value of the state variables on the interpolation points over the body surface (IP) and perform surface integration. In general, for what concerns the IB approach, none of the degrees of freedom where the solution is computed lies on the body surface; hence, interpolation is needed. We use the inverse distance weight to compute the state at each interpolation point:

$$Q_{IP} = \frac{\sum_i \frac{Q_i}{d_i}}{\sum_i \frac{1}{d_i}}, \tag{29}$$

where Q_{IP} is the state at the interpolation point, d_i is the distance between the interpolation point and the i^{th}-degree of freedom and Q_i is the state at the i^{th}-degree of freedom. Data reconstruction is performed starting from the so-called band region, i.e. the set of all degrees of freedom lying in the fluid region (Ω_f) close enough to the body surface. These degrees of freedom are candidates for being inside the set of points used for the interpolation. HORSES3D uses a nearest neighbour algorithm to find the n points (where n is a user-defined parameter) closer to the IP point and, once all the values of the IPs are known, integration is performed.

We studied in [129] the convergence of IB based on volume penalisation. It was shown that there is a loss of accuracy related to the loss of regularity of the solution near immersed boundaries, but that this degradation is local (approx. one element near the IB). To improve the convergence rates, one can impose a locally reconstructed gradient [132] or to perform local h/p-refinement near IB walls to improve the local resolution. In any case, the overall flow field benefits from using high order methods showing exponential convergence when we augment the polynomial order. In addition, in [128–130], we have proposed means for analysing the numerical errors arising from IB volume penalisations, and have proposed means for controlling these errors. In summary, using IB with high order methods is advantageous, but further developments are necessary to achieve high accuracy very close to walls. Note that the immersed boundary method is able to represent static and moving geometries in laminar or turbulent regimes. When turbulent boundary layers are simulated (by means of RANS or LES), we use wall functions to ameliorate lack of resolution near walls.

Fig. 14 shows how the IB method can be used to simulate a horizontal wind turbine (compressible solver with LES turbulent model). In this case the tower and nacelle are static, but the blades rotate with a constant rotational speed. It can be seen that the IB method captures the leading edge accelerations and blade tip vortices without the necessity of generating a complex body-fitted mesh. As an alternative for compute horizontal axis turbines, we have recently implemented various actuator line formulations.



(a) The STL geometry is included for post-processing clarity.

(b) The high-order mesh is shown. Near the rotor and wake a $P = 3$ polynomial is used, whilst $P = 2$ is selected far from the rotor and wake.

Fig. 14. Wind turbine simulation with immersed boundary method. Isosurface of velocity magnitude showing the tip vortex and the acceleration region at the blade leading edge.

5. Licensing, code structure, manuals and test cases

The code is maintained and available on GitHub. HORSES3D is a copyright of NUMATH https://numath.dmae.upm.es under the MIT License. NUMATH (Numerical Methods in Aerospace Technology) is a research group belonging to the School of Aeronautics in Madrid (Universidad Politécnica de Madrid). The objective of the group is to develop high-order numerical methods.

5.1. GitHub and continuous integration

The source code of HORSES3D is stored on GitHub. It has a number of test cases to check the various physics and numerical methods. The objective of test cases is to quickly check if a new commit "breaks" the code. To that end, once a new functionality is implemented, a test case is generated that checks it. The solution of this test case is stored and compared to that obtained with new versions of the code. The process of running the test cases and checking the solutions is integrated within the GitHub workflow framework. It is set to run automatically with a pull request trigger. Furthermore, a more complete test case suit is run on a monthly basis to check the code in more detail (e.g., compilation and run with different compilers in serial/parallel, etc.).

5.2. Documentation

HORSES3D includes a *doc* folder with documentation and user manuals where all parameters are identified. The solver uses control files, $*.control$, where the parameters and boundary conditions for the simulations are set.

A variety of test cases are included for the different sets of equations: Cahn-Hilliard, Euler, Incompressible Navier-Stokes, Multiphase, Navier-Stokes, Particles or Navier-Stokes coupled to Cahn-Hilliard. For example, for the compressible Navier-Stokes we include: ActuatorLine, Cylinder (body fitted and using immersed boundaries), EnergyConservingTest, FlatPlate, ForwardFacingStep, ManufacturedSolutions, NACA0012, TaylorGreen.

These cases are integrated into the Github workflow framework and run automatically (see previous section).

6. Future directions

We are constantly updating HORSES3D with new features, multiphysics and coupling it with new techniques. For example, we have recently linked HORSES3D to neural networks to accelerate simulations and have allowed the solver to call other external libraries through Python interfaces. Future directions include:

- Neural networks to accelerate high-order methods:
  High-order discontinuous Galerkin methods allow accurate solutions through the use of high-order polynomials inside each mesh element. Increasing the polynomial order leads to high accuracy, but increases the cost. On the one hand, high-order polynomials require more restrictive time-steps when using explicit temporal schemes, and on the other hand, the quadrature rules lead to more costly evaluations per iteration. In a recent work, we propose to accelerate high-order DG methods using Neural Networks. To this aim, we train a Neural Network using a high-order discretisation, to extract a corrective forcing that can be applied to a low order solution with the aim of recovering high-order accuracy. With this corrective forcing term, we can run a low-order solution (low cost) and correct the solution to obtain high-order accuracy; see details in [133,134].
- Coupling with other solvers (e.g., MFEM [135,136]) using Python:
  It is often advantageous to link flow solvers with external libraries (e.g., when performing fluid-structure interaction coupling or conjugate heat transfer). In these cases, external libraries can be called to perform part of the simulation (structural calculation or solving heat/radiation). To allow for this flexibility, we have developed an interface from Fortran to Python. Using this interface, it is possible to call Python routines at any point of the CFD calculation. Additionally, it is possible to call other libraries written in C/C++ from Python. An example of this approach can be found in [137], where we have coupled HORSES3D to MFEM to compute conjugate heat transfer through the Apollo capsule.

7. Conclusions

HORSES3D is an open-source parallel framework for the solution of non-linear partial differential equations. The solver allows the simulation of compressible flows (with and without shocks), incompressible flows, RANS and LES turbulence models, particle dynamics, multiphase flows, and aeroacoustics. The numerical schemes provide fast computations, especially when selecting high-order polynomials and MPI/OpenMP combinations for a large number of processors. Entropy-stable schemes, local anisotropic p-adaptation, and efficient dual time-stepping and multigrid allow the solver to tackle problems of industrial relevance, such as aircrafts and wind turbines.

The modularity and object-oriented programming architecture allow for easy integration of new physics and numerical methodologies, ensuring the necessary flexibility in a research focused solver.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Developer's repository link: https://github.com/loganoz/horses3d.

Acknowledgements

EF, GN and WL acknowledge the financial support of the European Union's Horizon 2020 research and innovation programme under the Marie Skłodowska-Curie grant agreement (MSCA ITN-EID-GA ASIMIA No 813605). GR and EV acknowledge the funding received by the Grant SIMOPAIR (Project No. RTI2018-097075-B-I00) funded by MCIN/AEI/10.13039/501100011033 and by ERDF A way of making Europe finances SIMOPAIR (Project No. RTI2018-097075-B-I00). OM and EV thank the European Union Horizon 2020 Research and Innovation Program under the Marie Sklodowska-Curie grant agreement No 860101 for the zEPHYR project. SC and EV thank the European Union Horizon 2020 Research and Innovation Program under the Marie Sklodowska-Curie grant agreement No 955923 for the Ssecoid project. DAK is supported by a grant from the Simons Foundation (#426393, David Kopriva). Andrés M. Rueda-Ramírez acknowledges funding through the Klaus-Tschira Stiftung via the project "HiFiLab". We furthermore thank the Regional Computing Center of the University of Cologne (RRZK) for providing computing time on the High Performance Computing (HPC) system ODIN as well as support. Esteban Ferrer would like to thank the support of the Spanish Ministry MCIN/AEI/10.13039/501100011033 and the European Union NextGenerationEU/PRTR finances "Europa Investigación 2020" EIN2020-112255, and also the Comunidad de Madrid finances Research Grants for Young Investigators from the Universidad Politécnica de Madrid.

Finally, all authors gratefully acknowledge the Universidad Politécnica de Madrid (www.upm.es) for providing computing resources on Magerit Supercomputer.

References

[1] Z. Wang, K. Fidkowski, R. Abgrall, F. Bassi, D. Caraeni, A. Cary, H. Deconinck, R. Hartmann, K. Hillewaert, H. Huynh, N. Kroll, et al., Int. J. Numer. Methods Fluids 72 (2013) 811.
[2] Z. Wang, Sci. China, Phys. Mech. Astron. 59 (2016) 1.
[3] M. Kompenhans, G. Rubio, E. Ferrer, E. Valero, J. Comput. Phys. 306 (2016) 216.
[4] M. Kompenhans, G. Rubio, E. Ferrer, E. Valero, Comput. Fluids 139 (2016) 36.
[5] A.M. Rueda-Ramírez, G. Rubio, E. Ferrer, E. Valero, J. Sci. Comput. 78 (2019) 433.
[6] A.M. Rueda-Ramírez, J. Manzanero, E. Ferrer, G. Rubio, E. Valero, J. Comput. Phys. 378 (2019) 209.
[7] D. Gottlieb, S.A. Orszag, Numerical Analysis of Spectral Methods: Theory and Applications, SIAM, 1977.
[8] W.H. Reed, T.R. Hill, Tech. Rep., Los Alamos Scientific Lab., N. Mex. (USA), 1973.
[9] F. Bassi, S. Rebay, J. Comput. Phys. 131 (1997) 267.
[10] F. Bassi, S. Rebay, G. Mariotti, S. Pedinotti, M. Savini, in: Proceedings of the 2nd European Conference on Turbomachinery Fluid Dynamics and Thermodynamics, Antwerpen, Belgium, 1997, pp. 99–109.
[11] P. Fischer, J. Kruse, J. Mullen, H. Tufo, J. Lottes, S. Kerkemeier, Argonne National Laboratory, Mathematics and Computer Science Division, Argonne, IL, 2008.
[12] C. Cantwell, D. Moxey, A. Comerford, A. Bolis, G. Rocco, G. Mengaldo, D. De Grazia, S. Yakovlev, J.-E. Lombard, D. Ekelschot, et al., Comput. Phys. Commun. (ISSN 0010-4655) 192 (2015) 205, https://www.sciencedirect.com/science/article/pii/S0010465515000533.
[13] D. Moxey, C.D. Cantwell, Y. Bao, A. Cassinelli, G. Castiglioni, S. Chun, E. Juda, E. Kazemi, K. Lackhove, J. Marcon, et al., Comput. Phys. Commun. (ISSN 0010-4655) 249 (2020) 107110, https://www.sciencedirect.com/science/article/pii/S0010465519304175.
[14] H. Blackburn, D. Lee, T. Albrecht, J. Singh, Comput. Phys. Commun. (ISSN 0010-4655) 245 (2019) 106804, https://www.sciencedirect.com/science/article/pii/S0010465519301730.
[15] W. Bangerth, R. Hartmann, G. Kanschat, ACM Trans. Math. Softw. (ISSN 0098-3500) 33 (2007), https://doi.org/10.1145/1268776.1268779, 24–es.
[16] G.J. Gassner, A.R. Winters, D.A. Kopriva, J. Comput. Phys. (ISSN 0021-9991) 327 (2016) 39, https://www.sciencedirect.com/science/article/pii/S0021999116304259.
[17] F. Hindenlang, G.J. Gassner, C. Altmann, A. Beck, M. Staudenmaier, C.-D. Munz, in: "High Fidelity Flow Simulations", Onera Scientific Day, Comput. Fluids (ISSN 0045-7930) 61 (2012) 86, https://www.sciencedirect.com/science/article/pii/S004579301200093X.
[18] H. Ranocha, M. Schlottke-Lakemper, A.R. Winters, E. Faulhaber, J. Chan, G. Gassner, Proc. JuliaCon Conf. 1 (2022) 77, arXiv:2108.06476, 2022.
[19] M. Schlottke-Lakemper, A.R. Winters, H. Ranocha, G.J. Gassner, J. Comput. Phys. 442 (2021) 110467, arXiv:2008.10593.
[20] F. Witherden, A. Farrington, P. Vincent, Comput. Phys. Commun. (ISSN 0010-4655) 185 (2014) 3028, https://www.sciencedirect.com/science/article/pii/S0010465514002549.
[21] C. Geuzaine, J.-F. Remacle, Int. J. Numer. Methods Eng. 79 (2009) 1309.
[22] M. Folk, A. Cheng, K. Yates, in: Proceedings of Supercomputing, vol. 99, 1999, pp. 5–33.
[23] D.A. Kopriva, et al., HOHQMesh, the high order hex-quad mesher, https://github.com/trixi-framework/HOHQMesh.
[24] F. Hindenlang, T. Bolemann, C.-D. Munz, in: IDIHOM: Industrialization of High-Order Methods-A Top-Down Approach, Springer, 2015, pp. 133–152.
[25] J. Ahrens, B. Geveci, C. Law, in: The Visualization Handbook, vol. 717, 2005.
[26] U. Ayachit, The Paraview Guide: a Parallel Visualization Application, Kitware, Inc., 2015.
[27] D.A. Kopriva, Implementing Spectral Methods for Partial Differential Equations, Springer, Netherlands, 2009.
[28] J. Manzanero, G. Rubio, D.A. Kopriva, E. Ferrer, E. Valero, J. Comput. Phys. (2020) 109363.
[29] E. Ferrer, J. Comput. Phys. (ISSN 0021-9991) 348 (2017) 754, http://www.sciencedirect.com/science/article/pii/S0021999117305570.
[30] E. Toro, Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction, Springer Berlin Heidelberg, ISBN 9783540498346, 2009, URL https://books.google.se/books?id=SqEjX0um8o0C.
[31] D. Arnold, F. Brezzi, B. Cockburn, L. Marini, SIAM J. Numer. Anal. 39 (2001) 1749.
[32] E. Ferrer, Ph.D. thesis, Oxford University, UK, 2012.
[33] E. Ferrer, R. Willden, in: 10th ICFD Conference Series on Numerical Methods for Fluid Dynamics (ICFD 2010), Comput. Fluids (ISSN 0045-7930) 46 (2011) 224, https://www.sciencedirect.com/science/article/pii/S0045793010002860.
[34] E. Ferrer, R.H. Willden, J. Comput. Phys. (ISSN 0021-9991) 231 (2012) 7037, https://www.sciencedirect.com/science/article/pii/S0021999112002215.
[35] A. Beck, T. Bolemann, D. Flad, H. Frank, G. Gassner, F. Hindenlang, C. Munz, Int. J. Numer. Methods Fluids 76 (2014) 522.
[36] T.C. Fisher, M.H. Carpenter, J. Comput. Phys. 252 (2013) 518.
[37] M.H. Carpenter, T.C. Fisher, E.J. Nielsen, S.H. Frankel, SIAM J. Sci. Comput. 36 (2014) B835.
[38] J. Manzanero, G. Rubio, E. Ferrer, E. Valero, D.A. Kopriva, J. Sci. Comput. 75 (2018) 1262, https://doi.org/10.1007/s10915-017-0585-6.
[39] D.A. Kopriva, G.J. Gassner, SIAM J. Sci. Comput. 36 (2014) A2076.
[40] G.J. Gassner, A.R. Winters, D.A. Kopriva, J. Comput. Phys. 327 (2016) 39.
[41] G.J. Gassner, A.R. Winters, F.J. Hindenlang, D.A. Kopriva, J. Sci. Comput. 77 (2018) 154.

[42] T. Chen, C.-W. Shu, J. Comput. Phys. 345 (2017) 427.
[43] A.R. Winters, R.C. Moura, G. Mengaldo, G.J. Gassner, S. Walch, J. Peiro, S.J. Sherwin, J. Comput. Phys. 372 (2018) 1.
[44] G.J. Gassner, SIAM J. Sci. Comput. 35 (2013) A1233.
[45] A.R. Winters, D.A. Kopriva, G.J. Gassner, F. Hindenlang, in: Efficient High-Order Discretizations for Computational Fluid Dynamics, Springer, 2021, pp. 117–196.
[46] T. Chen, C.-W. Shu, CSIAM Trans. Appl. Math. 1 (2020) 1.
[47] Y. Morinishi, J. Comput. Phys. 229 (2010) 276.
[48] F. Ducros, F. Laporte, T. Soulères, V. Guinot, P. Moinat, B. Caruelle, J. Comput. Phys. (ISSN 0021-9991) 161 (2000) 114, https://www.sciencedirect.com/science/article/pii/S0021999100964921.
[49] C.A. Kennedy, A. Gruber, J. Comput. Phys. 227 (2008) 1676.
[50] S. Pirozzoli, J. Comput. Phys. 229 (2010) 7180.
[51] P. Chandrashekar, Commun. Comput. Phys. 14 (2013) 1252–1286.
[52] P. Chandrashekar, J. Comput. Phys. (ISSN 0021-9991) 233 (2013) 527, https://www.sciencedirect.com/science/article/pii/S0021999112005487.
[53] J. Manzanero, G. Rubio, D.A. Kopriva, E. Ferrer, E. Valero, J. Comput. Phys. 408 (2020) 109241.
[54] J.-L. Guermond, L. Quartapelle, J. Comput. Phys. 165 (2000) 167.
[55] D. Lodares, J. Manzanero, E. Ferrer, E. Valero, J. Comput. Phys. (ISSN 0021-9991) 455 (2022) 110998, https://www.sciencedirect.com/science/article/pii/S0021999122000602.
[56] G. Ntoukas, J. Manzanero, G. Rubio, E. Valero, E. Ferrer, J. Comput. Phys. 458 (2022) 111093.
[57] D.A. Kopriva, F.J. Hindenlang, T. Bolemann, G.J. Gassner, J. Sci. Comput. 79 (2019) 1389.
[58] D.A. Kopriva, S.L. Woodruff, M.Y. Hussaini, Int. J. Numer. Methods Eng. 53 (2002) 105.
[59] C. Roy, in: 47th AIAA Aerospace Sciences Meeting Including the New Horizons Forum and Aerospace Exposition, 2009, p. 1302.
[60] F. Fraysse, G. Rubio, J. De Vicente, E. Valero, Aerosp. Sci. Technol. 38 (2014) 76.
[61] G. Rubio, F. Fraysse, J. de Vicente, E. Valero, J. Sci. Comput. 57 (2013) 146.
[62] G. Rubio, F. Fraysse, D.A. Kopriva, E. Valero, J. Sci. Comput. 64 (2015) 425.
[63] W. Laskowski, G. Rubio, E. Valero, E. Ferrer, J. Comput. Phys. 451 (2022) 110883.
[64] G. Ntoukas, J. Manzanero, G. Rubio, E. Valero, E. Ferrer, J. Comput. Phys. 442 (2021) 110409.
[65] J. Williamson, J. Comput. Phys. 35 (1980) 48.
[66] M.H. Carpenter, C.A. Kennedy, NASA Report TM 109112, NASA Langley Research Center, 1994.
[67] S. Joshi, A. Hurtado-de Mendoza, J. Kou, K. Puri, C. Hirsch, E. Ferrer, in: WCCM-ECCOMAS2020, 2021.
[68] B. Vermeire, N. Loppi, P. Vincent, J. Comput. Phys. (ISSN 0021-9991) 383 (2019) 55.
[69] K.J. Fidkowski, T.A. Oliver, J. Lu, D.L. Darmofal, J. Comput. Phys. (ISSN 0021-9991) 207 (2005) 92.
[70] M. Parsani, K. Van den Abeele, C. Lacor, E. Turkel, J. Comput. Phys. 229 (2010) 828.
[71] A. Ghidoni, A. Colombo, F. Bassi, S. Rebay, Int. J. Numer. Methods Fluids 75 (2014) 134.
[72] F. Bassi, L. Botti, A. Colombo, A. Ghidoni, F. Massa, Comput. Fluids 118 (2015) 305.
[73] T.F. Coleman, J.J. Moré, SIAM J. Numer. Anal. 20 (1983) 187.
[74] A.H. Gebremedhin, F. Manne, A. Pothen, SIAM Rev. 47 (2005) 629.
[75] G.I. Taylor, A.E. Green, Proc. R. Soc. A, Math. Phys. Eng. Sci. 158 (1937) 499–521.
[76] R.C. Moura, G. Mengaldo, J. Peiró, S.J. Sherwin, in: Spectral and High Order Methods for Partial Differential Equations ICOSAHOM 2016, Springer, 2017, pp. 161–173.
[77] J. Manzanero, E. Ferrer, G. Rubio, E. Valero, Comput. Fluids (ISSN 0045-7930) 200 (2020) 104440, https://www.sciencedirect.com/science/article/pii/S0045793020300165.
[78] BSC-CNS, Marenostrum web page, https://www.bsc.es/marenostrum/marenostrum/mn3.
[79] W. Laskowski, Ph.D. thesis, ETSIAE-UPM, 2022.
[80] P. Sagaut, Large Eddy Simulation for Incompressible Flows: an Introduction, Springer Science & Business Media, 2006.
[81] T.A. Oliver, Tech. Rep., Massachusetts Inst of Tech Cambridge Dept of Aeronautics and Astronautics, 2008.
[82] J. Vassberg, M. Dehaan, M. Rivers, R. Wahls, in: 26th AIAA Applied Aerodynamics Conference, 2008, p. 6919.
[83] M.A. de Barros Ceze, Ph.D. thesis, University of Michigan, 2013.
[84] J. Smagorinsky, Mon. Weather Rev. 91 (1963) 99.
[85] D.K. Lilly, Mon. Weather Rev. 93 (1965) 11.
[86] F. Nicoud, F. Ducros, Flow Turbul. Combust. 62 (1999) 183.
[87] A. Vreman, Phys. Fluids 16 (2004) 3670.
[88] F.F. Grinstein, L.G. Margolin, W.J. Rider, Implicit Large Eddy Simulation, vol. 10, Cambridge University Press, Cambridge, 2007.
[89] G. Gassner, D.A. Kopriva, SIAM J. Sci. Comput. 33 (2011) 2560, https://doi.org/10.1137/100807211.
[90] J. Kou, S. Le Clainche, E. Ferrer, J. Comput. Phys. (ISSN 0021-9991) 449 (2022) 110798, https://www.sciencedirect.com/science/article/pii/S0021999121006938.
[91] J. Manzanero, E. Ferrer, G. Rubio, E. Valero, arXiv:1805.10519, 2018.
[92] J. Manzanero, G. Rubio, E. Ferrer, E. Valero, SIAM J. Sci. Comput. 40 (2018) A747.
[93] P. Solán-Fustero, A. Navas-Montilla, E. Ferrer, J. Manzanero, P. García-Navarro, J. Comput. Phys. (ISSN 0021-9991) 435 (2021) 110246, https://www.sciencedirect.com/science/article/pii/S0021999121001418.
[94] D. Flad, G. Gassner, J. Comput. Phys. (ISSN 0021-9991) 350 (2017) 782, http://www.sciencedirect.com/science/article/pii/S002199911730654X.
[95] A. Uranga, P. Persson, M. Drela, J. Peraire, Int. J. Numer. Methods Eng. (ISSN 1097-0207) 87 (2011) 232, https://doi.org/10.1002/nme.3036.
[96] E. Ferrer, J. Manzanero, A.M. Rueda-Ramirez, G. Rubio, E. Valero, in: S.J. Sherwin, D. Moxey, J. Peiró, P.E. Vincent, C. Schwab (Eds.), Spectral and High Order Methods for Partial Differential Equations ICOSAHOM 2018, Springer International Publishing, Cham, ISBN 978-3-030-39647-3, 2020, pp. 477–487.
[97] P. Fernandez, N. Nguyen, J. Peraire, J. Comput. Phys. (ISSN 0021-9991) 336 (2017) 308, https://www.sciencedirect.com/science/article/pii/S0021999117301080.
[98] E. Ferrer, N. Saito, H. Blackburn, D. Pullin, Comput. Fluids (ISSN 0045-7930) 191 (2019) 104239, https://www.sciencedirect.com/science/article/pii/S0045793018305875.
[99] J. Shen, in: Proceedings of the 1994 Beijing Symposium on Nonlinear Evolution Equations and Infinite Dynamical Systems, ZhongShan University Press, Zhongshan, 1997, pp. 68–78.
[100] G. Karniadakis, S. Sherwin, Spectral/hp Element Methods for Computational Fluid Dynamics, Oxford Science Publications, 2005.
[101] E. Ferrer, D. Moxey, R.H.J. Willden, S.J. Sherwin, Commun. Comput. Phys. 16 (2014) 817–840.
[102] J.W. Cahn, J.E. Hilliard, J. Chem. Phys. 28 (1958) 258.
[103] S.M. Allen, J.W. Cahn, Acta Metall. 20 (1972) 423.
[104] J. Manzanero, G. Rubio, D.A. Kopriva, E. Ferrer, E. Valero, J. Comput. Phys. 403 (2020) 109072.
[105] J. Manzanero, C. Redondo, G. Rubio, E. Ferrer, Ángel Rivero–Jiménez, Comput. Fluids (ISSN 0045-7930) 225 (2021) 104971, https://www.sciencedirect.com/science/article/pii/S0045793021001389.
[106] R. von Mises, J. Aeronaut. Sci. 17 (1950) 551.
[107] J.L. Guermond, B. Popov, SIAM J. Appl. Math. 74 (2014) 284.
[108] E. Tadmor, SIAM J. Numer. Anal. 26 (1989) 30.
[109] Y. Maday, S. Ould Kaber, E. Tadmor, SIAM J. Numer. Anal. 30 (1993) 321.
[110] R. Moura, S. Sherwin, J. Peiro, J. Comput. Phys. 307 (2016) 401.
[111] K.O. Friedrichs, P.D. Lax, Proc. Natl. Acad. Sci. USA 68 (1971) 1686.
[112] E. Tadmor, Appl. Numer. Math. 2 (1986) 211.
[113] A. Mateo-Gabín, J. Manzanero, E. Valero, J. Comput. Phys. 471 (2022) 111618.
[114] A. Frankel, H. Pouransari, F. Coletti, A. Mani, J. Fluid Mech. 792 (2016) 869.
[115] H. Pouransari, A. Mani, J. Sol. Energy Eng. 139 (2017).
[116] S.K. Lele, J.W. Nichols, Trans. R. Soc. A 372 (2014).
[117] J.E.F. Williams, D.L. Hawkings, Philos. Trans. R. Soc. Lond. Ser. A, Math. Phys. Sci. 264 (1969) 321.
[118] A. Najafi-Yazdi, G.A. Brès, L. Mongeau, Proc. R. Soc. A, Math. Phys. Eng. Sci. 467 (2010) 144.
[119] G. Ghorbaniasl, C. Lacor, J. Sound Vib. 331 (2012) 117.
[120] I.E. Garrick, C.E. Watkins, I.E. Garrick, C.E. Watkins, Tech. Rep., 1954.
[121] F. Farassat, Tech. Rep., 2007.
[122] K.S. Brentner, F. Farassat, Prog. Aerosp. Sci. 39 (2003) 83.
[123] D. Lockard, in: 8th AIAA/CEAS Aeroacoustics Conference Exhibit, American Institute of Aeronautics and Astronautics, 2002.
[124] R.W. Paterson, P.G. Vogt, M.R. Fink, C.L. Munch, J. Aircr. 10 (1973) 296, https://doi.org/10.2514/3.60229.
[125] G. Desquesnes, M. Terracol, P. Sagaut, J. Fluid Mech. 591 (2007) 155, https://doi.org/10.1017/s0022112007007896.
[126] C.S. Peskin, J. Comput. Phys. (ISSN 0021-9991) 10 (1972) 252, https://www.sciencedirect.com/science/article/pii/0021999172900654.
[127] W. Kim, H. Choi, Int. J. Heat Fluid Flow (ISSN 0142-727X) 75 (2019) 301, https://www.sciencedirect.com/science/article/pii/S0142727X1830691X.
[128] J. Kou, A.H. de Mendoza, S. Joshi, S. Le Clainche, E. Ferrer, J. Comput. Phys. (ISSN 0021-9991) 449 (2022) 110817, https://www.sciencedirect.com/science/article/pii/S0021999121007129.
[129] J. Kou, S. Joshi, A.H. de Mendoza, K. Puri, C. Hirsch, E. Ferrer, J. Comput. Phys. (ISSN 0021-9991) 448 (2022) 110721, https://www.sciencedirect.com/science/article/pii/S0021999121006161.
[130] J. Kou, E. Ferrer, J. Comput. Phys. (ISSN 0021-9991) 472 (2023) 111678, https://www.sciencedirect.com/science/article/pii/S0021999122007410.
[131] Z. Wu, F. Zhao, X. Liu, in: Proceedings of the ACM SIGGRAPH Symposium on High Performance Graphics, 2011, pp. 71–78.
[132] A.E. Giannenas, N. Bempedelis, F.N. Schuch, S. Laizet, Flow Turbul. Combust. 109 (2022) 931.

[133] F. Manrique de Lara, E. Ferrer, Comput. Fluids (ISSN 0045-7930) 235 (2022) 105274, https://www.sciencedirect.com/science/article/pii/S0045793021003698.
[134] F. Manrique de Lara, E. Ferrer, arXiv preprint, arXiv:2207.11571, 2022.
[135] mfem, MFEM: modular finite element methods [software], mfem.org.
[136] R. Anderson, J. Andrej, A. Barker, J. Bramwell, J.-S. Camier, J.C.V. Dobrev, Y. Dudouit, A. Fisher, T. Kolev, W. Pazner, et al., Comput. Math. Appl. 81 (2021) 42.
[137] D.H. Perea, https://oa.upm.es/71808/, 2022.